CS336 作业 2：系统与并行计算

版本 26.1.3 · CS336 课程组 · 2026 年春季

译文说明：本译文对应用户提供的 48 页英文 PDF，完整保留章节、题目标识、分值和提交要求。公式、代码、运行输出及示意图沿用原文图块，以保留精确的符号、接口及格式；算法另附中文逐行说明。文中的“提交内容”对应 Deliverable。此文件是作业说明的翻译，不包含习题解答。原文中值得注意的表述差异以“译注”单独标明。

内容导航

1 作业概述

2 性能分析与基准测试

3 单 GPU 内存

4 GPU 内核与 FlashAttention-2

5 分布式数据并行训练

6 优化器状态分片

7 全分片数据并行

8 并行策略分析

9 排行榜

参考文献

阅读提示：PDF 书签可用于章节跳转。代码、公式和图示下方标有原文页码，方便与英文版本对照。

1 作业概述

本次作业将带你动手提升单 GPU 训练速度，并将训练扩展到多 GPU。

你将实现的内容

1. 基准测试与性能分析框架。

2. 激活检查点（activation checkpointing）。

3. 使用 Triton 编写的 FlashAttention-2 内核。

4. 分布式数据并行训练。

5. 优化器状态分片。

6. 全分片数据并行训练。

代码结构

作业代码与英文说明位于 GitHub：https://github.com/stanford-cs336/assignment2-systems 。请使用 Git 克隆仓库。如果有更新，我们会通知你，届时可通过 git pull 获取最新版。

1. cs336-basics/：本作业需要分析作业 1 中部分组件的性能。该目录包含课程组的作业 1 参考解答，其中有 cs336-basics/pyproject.toml 和 cs336-basics/cs336_basics 包。如果想使用自己实现的模型，可修改根目录中的 pyproject.toml，使其指向你自己的包。

2. /：cs336-systems 的根目录。我们创建了一个名为 cs336_systems 的空模块，其中没有代码，你可以从零开始自由实现。

3. tests/*.py：包含你必须通过的全部测试。测试会调用 tests/adapters.py 中定义的钩子；你需要实现适配器，把自己的代码接入测试。新增测试或修改测试代码可以帮助调试，但最终实现必须通过原始测试套件。

4. README.md：包含预期目录结构的更多说明，以及环境配置的基本操作。

如何提交

向 Gradescope 提交以下文件：

• writeup.pdf：回答所有书面问题。请使用排版工具编写答案。

• code.zip：包含你编写的全部代码。

运行 test_and_make_submission.sh 脚本生成 code.zip。

2 性能分析与基准测试

作业第一部分将探索如何优化 Transformer 模型的性能，以便更高效地利用 GPU。我们会分析模型在前向和反向传播中把时间与内存花在了哪里，再用自定义 GPU 内核优化自注意力，使其运行速度超过普通 PyTorch 实现所能达到的水平。后续部分将使用多个 GPU，并学习如何在集群上训练模型。

2.1 性能分析

在着手优化之前，先分析程序如何消耗时间、内存等资源十分有益。否则，我们可能优化了本来就不占多少时间或内存的部分，最终却看不到可测量的端到端收益。

我们将实现三种性能评估方式：

1. 使用 Python 标准库为前向和反向传播计时，进行简单的端到端基准测试。

2. 使用 NVIDIA Nsight Systems 分析计算性能，了解时间如何分布在 CPU 和 GPU 的各项操作上。

3. 分析内存使用情况。

2.1.1 准备工作：导入 basics Transformer 模型

先确保能够加载上一次作业的模型。作业 1 已将模型组织成 Python 包，以便后续导入。我们在 ./cs336-basics 中添加了模型参考实现，并在 pyproject.toml 中指定了该路径。照常使用 uv run [command]，uv 就会自动找到本地 cs336-basics 包。如果想使用自己的模型实现，可修改 pyproject.toml 指向自己的包。

通过以下命令检查能否导入模型：

```
$ uv run python
Using CPython 3.13.13
Creating virtual environment at: /path/to/uv/env/dir
      Built cs336-systems @ file:///path/to/systems/dir
      Built cs336-basics @ file:///path/to/basics/dir
Installed 78 packages in 168ms
Python 3.13.13 (main, Apr  7 2026, 20:49:46) [Clang 22.1.1 ] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> import cs336_basics
...
```

原文第 2 页 · 原式 / 代码 / 图示

现在应该可以使用作业 1 的相关模块了。例如，model.py 可通过 import cs336_basics.model 导入。

2.1.2 模型规模

本作业会对不同模型进行基准测试和性能分析，以了解其性能。为观察规模扩大后的变化，我们将使用并引用下表中的配置。除排行榜任务外，所有模型的词表大小均为 10,000，批大小均为 4，上下文长度则会变化。本作业及后续作业需要用大量表格和图形展示结果。强烈建议用代码自动生成报告中的表格，因为手动在 LaTeX 或 Typst 中排表可能十分繁琐。可以使用 pandas.DataFrame.to_latex()、pandas.DataFrame.to_typst()，也可以从自己喜欢的表格表示形式编写生成函数。

规模	d_model	d_ff	num_layers	num_heads
small	768	3072	12	12
medium	1024	4096	24	16
large	1280	5120	36	20
xl	2560	10240	32	32
10B	4608	12288	50	36

表 1：不同模型规模的参数，主要参考 GPT-2 配置。除非另有说明，上下文长度统一使用 512。

2.1.3 端到端基准测试

现在实现一个简单的性能评估脚本。之后我们会测试模型的许多变体，例如改变精度、替换层等，因此建议通过命令行参数控制这些变化，方便后续实验。

先进行最简单的分析：为前向传播、反向传播和优化器更新步骤计时。因为只测量速度和内存，使用随机权重和随机数据即可。

性能测量需要谨慎，一些常见陷阱会导致测到的并不是我们想要的量。对 GPU 代码而言，一个关键点是 CUDA 调用是异步的。调用 torch.matmul 等 CUDA 内核时，PyTorch 函数不会等待矩阵乘法完成，就会把控制权返回给调用代码。这样 GPU 执行矩阵乘法时，CPU 可以继续运行并调度新操作，从而显著提升性能。但这也意味着，仅测量 torch.matmul 调用多久返回，并不能知道 GPU 实际执行矩阵乘法用了多久。

在 PyTorch 中，可以调用 torch.cuda.synchronize() 等待所有已调度的 GPU 内核完成，以更准确地测量 CUDA 内核运行时间。这里的“同步”是让 CPU 运行时与 GPU 运行时同步。基于这一点，下面编写基础性能分析设施。

题目 benchmarking_script：基准测试脚本（4 分）

(a) 编写脚本，对模型的前向传播、反向传播和优化器更新进行基本端到端基准测试。具体支持：

• 根据给定超参数（例如层数）初始化模型。

• 生成一个随机数据批次。

• 在开始计时前运行 w 个预热步骤，然后测量 n 个步骤的执行时间。通过参数选择仅前向、前向加反向，或前向加反向再加优化器更新。可使用 Python 的 timeit 模块计时，例如 timeit 函数或 timeit.default_timer()。后者提供系统中分辨率最高的时钟，因此比 time.time() 更适合用作基准测试的默认计时器。

• 每个步骤之后调用 torch.cuda.synchronize()。

提交内容：一个脚本，根据给定超参数初始化 basics Transformer 模型，创建随机数据批次，并测量仅前向、前向加反向，以及包含优化器更新的完整训练步骤。

(b) 对第 2.1.2 节中各个规模的模型，测量前向、反向和优化器更新的时间。预热 5 步，再测量 10 步并计算均值与标准差。一次前向传播耗时多久？反向传播呢？不同次测量之间波动很大，还是标准差很小？

提交内容：用 1-2 句话回答，并给出测量时间。

(c) 基准测试的一个常见问题是没有预热。不进行预热，重复上述分析。结果受到什么影响？你认为为什么会这样？再尝试只预热 1 步或 2 步。为什么结果仍可能不同？

提交内容：用 2-3 句话回答。

2.1.4 Nsight Systems 性能分析工具

端到端基准测试无法告诉我们模型的前向和反向传播中，时间与内存具体花在了哪里，因此不能揭示具体的优化机会。要了解程序每个组件（如函数）的耗时，可以使用 profiler。执行性能分析器会在函数开始和结束时插入监测代码，从而提供函数级的详细统计，例如调用次数、平均耗时和累计耗时。

普通 Python 性能分析器（如 CProfile）不能分析 CUDA 内核，因为这些内核在 GPU 上异步执行。NVIDIA 提供了可以通过命令行 nsys 使用的分析工具。建议从包管理器或其下载页面获取较新版本：https://developer.nvidia.com/nsight-systems/get-started 。本部分将使用 nsys 分析 Transformer 模型的运行时间。

nsys 的用法很直接：在上一节 Python 脚本的启动命令前加上 nsys profile。例如，对 benchmark.py 进行基本分析：

```
$ uv run nsys profile -- python benchmark.py
```

原文第 4 页 · 原式 / 代码 / 图示

之后可以用本地 NVIDIA Nsight Systems 桌面应用打开分析结果：https://developer.nvidia.com/nsight-systems 。在轨迹的 CUDA API 行中选择某个 CPU 侧的 CUDA API 调用，会高亮 CUDA HW 行中对应的所有 GPU 内核执行。

更完整的一次分析可以使用以下命令：

```
$ uv run nsys profile --trace=cuda,cudnn,cublas,osrt,nvtx --pytorch=functions-trace,autograd-shapes-nvtx --cudabacktrace=all --python-backtrace=cuda --gpu-metrics-devices=0 -- python benchmark.py
```

原文第 4 页 · 原式 / 代码 / 图示



原文第 5 页 · 原式 / 代码 / 图示

图 1：详细的 Nsight Systems 执行轨迹。

上述命令中，--trace 指定要记录的 API；--pytorch 在模块调用与自动求导过程中插入 NVTX 标签；--cudabacktrace 和 --python-backtrace 提供更详细的调用栈，帮助定位代码中发起内核调用的位置；--gpu-metrics-devices 指定要测量哪块 GPU 的利用率。

性能分析本身会带来开销，使程序整体运行变慢。通常只启用当前关注的功能更划算。尤其是不需要调用栈时，可以去掉 --cudabacktrace=all 和 --python-backtrace=cuda，因为它们的开销特别大。

建议尝试 nsys profile 的不同命令行选项，了解它的功能；用户指南：https://docs.nvidia.com/nsight-systems/UserGuide/index.html 。你还可以用 NVTX range 标记代码区域，这些区域会在轨迹的 NVTX 行中显示为区块，涵盖对应的 CUDA API 调用与内核执行。特别是，应使用 NVTX range 将基准测试脚本中的预热步骤排除在测量之外：通过 --nvtx-capture 对相应 NVTX 标签进行筛选。

同样，你可以区分模型前向与反向传播分别对应哪些内核，甚至可以按以下方式标记自注意力实现，识别自注意力层不同部分对应的内核：

```
...
import torch.cuda.nvtx as nvtx

@nvtx.range("scaled dot product attention")
def annotated_scaled_dot_product_attention(
    ... # Q, K, V, mask
):
    ...
    with nvtx.range("computing attention scores"):
        ... # compute attention scores between Q and K

    with nvtx.range("computing softmax"):
        ... # compute softmax of attention scores

    with nvtx.range("final matmul"):
        ... # compute output projection
```

原文第 5 页 · 原式 / 代码 / 图示

```
    return ...
```

原文第 6 页 · 原式 / 代码 / 图示

在基准测试脚本中，可用带标记的版本替换原实现：

```
cs336_basics.model.scaled_dot_product_attention = annotated_scaled_dot_product_attention
```

原文第 6 页 · 原式 / 代码 / 图示

最后要注意，torch.compile 可能使时间和资源消耗难以归因到具体代码位置。你可能需要在代码不同部分添加或移除 torch.compile 包装及 NVTX 标记，才能正确地把耗时与资源使用对应到源代码。

题目 nsys_profile：Nsight Systems 性能分析（5 分）

从表 1 中自行选择两种模型规模，再选择三种大于 128、且为 2 的幂的上下文长度；其中最大的上下文长度应为内存能够容纳的最长长度。选择你认为最值得研究的组合，使用 nsys 分析前向、反向和优化器更新。对每份分析结果，回答：

(a) 前向传播的总耗时是多少？是否与之前用 Python 标准库测得的结果一致？

提交内容：用 1-2 句话回答。

(b) 前向传播中，哪个 CUDA 内核的累计 GPU 时间最长？模型单次前向传播会调用它多少次？同时执行前向和反向传播时，耗时最多的是否仍是同一个内核？提示：查看 Stats System View 中的 CUDA GPU Kernel Summary，并用 NVTX range 筛选，识别模型的哪些部分对应哪些内核。

提交内容：用 1-2 句话回答。

(c) 虽然绝大多数浮点运算发生在矩阵乘法中，你会发现其他一些内核也占据了不可忽视的总运行时间。除矩阵乘法外，前向传播中还有哪些内核占用了明显的 CUDA 运行时间？

提交内容：用 1-2 句话回答。

(d) 使用你实现的 AdamW 分析一个完整训练步骤：前向传播、计算损失、反向传播，最后进行优化器更新，流程与实际训练相同。相较于仅前向的推理，矩阵乘法耗时占比如何变化？其他内核呢？

提交内容：用 1-2 句话回答。

(e) 比较前向传播中自注意力层的 softmax 与矩阵乘法耗时。两者的耗时差异与 FLOPs 差异是否相称？

提交内容：用 1-2 句话回答。

2.1.5 混合精度

到目前为止，我们使用的都是 FP32：所有模型参数与激活的数据类型均为 torch.float32。不过，现代 NVIDIA GPU 包含专门加速低精度矩阵乘法的 Tensor Core。例如，NVIDIA B200 规格表列出的 FP32 峰值吞吐量为 80 TFLOPS，而 FP16（半精度浮点）或 BF16（bfloat16）的峰值吞吐量则高达 2500 TFLOPS。因此，使用低精度数据类型应能加速训练和推理。

但直接把模型转换为低精度可能降低模型精度。例如，实际中许多梯度太小，无法用 FP16 表示，直接用 FP16 训练时就会变成零。为此，FP16 训练通常使用损失缩放（loss scaling）：将损失乘以一个缩放系数，增大梯度幅值，避免下溢为零。此外，FP16 的动态范围小于 FP32，可能发生溢出，表现为损失出现 NaN。全程使用 bfloat16 通常更稳定，因为 BF16 与 FP32 的动态范围相同，但最终模型效果仍可能受到影响。

为了获得低精度的速度优势，实践中通常使用混合精度训练。PyTorch 通过 torch.autocast 上下文管理器实现这一点：矩阵乘法等操作以低精度执行，而累加、归约等需要 FP32 完整动态范围的操作保持高精度。以下代码会在前向传播中自动识别适合低精度的操作，并转换为指定类型：

```
model : torch.nn.Module = ... # e.g. your Transformer model
dtype : torch.dtype = ... # e.g. torch.bfloat16
x : torch.Tensor = ... # input data

with torch.autocast(device_type="cuda", dtype=dtype):
    y = model(x)
```

原文第 7 页 · 原式 / 代码 / 图示

如前所述，即使参与累加的张量本身已转为低精度，累加操作通常仍应保持高精度。下面的练习有助于理解原因。

题目 mixed_precision_accumulation：混合精度累加（1 分）

运行以下代码，并评价结果的准确性。

```
s = torch.tensor(0,dtype=torch.float32)
for i in range(1000):
    s += torch.tensor(0.01,dtype=torch.float32)
print(s)

s = torch.tensor(0,dtype=torch.float16)
for i in range(1000):
    s += torch.tensor(0.01,dtype=torch.float16)
print(s)

s = torch.tensor(0,dtype=torch.float32)
for i in range(1000):
```

原文第 7 页 · 原式 / 代码 / 图示

```
        s += torch.tensor(0.01,dtype=torch.float16)
    print(s)

    s = torch.tensor(0,dtype=torch.float32)
    for i in range(1000):
        x = torch.tensor(0.01,dtype=torch.float16)
        s += x.type(torch.float32)
    print(s)
```

原文第 8 页 · 原式 / 代码 / 图示

提交内容：用 2-3 句话回答。

接下来，先在一个简单模型中应用混合精度以建立直觉，再将其加入基准测试脚本。

题目 benchmarking_mixed_precision：混合精度基准测试（2 分）

(a) 考虑以下模型：

```
class ToyModel(nn.Module):
    def __init__(self, in_features: int, out_features: int):
        super().__init__()
        self.fc1 = nn.Linear(in_features, 10, bias=False)
        self.ln = nn.LayerNorm(10)
        self.fc2 = nn.Linear(10, out_features, bias=False)
        self.relu = nn.ReLU()

    def forward(self, x):
        x = self.relu(self.fc1(x))
        x = self.ln(x)
        x = self.fc2(x)
        return x
```

原文第 8 页 · 原式 / 代码 / 图示

假设模型在 GPU 上训练，参数最初为 FP32，现在希望使用 FP16 的自动混合精度。以下各项分别是什么数据类型？

- autocast 上下文中的模型参数；
- 第一个前馈层 ToyModel.fc1 的输出；
- 层归一化 ToyModel.ln 的输出；
- 模型预测的 logits；
- 损失；
- 模型梯度。

提交内容：列出以上各项的数据类型。

(b) 你应已观察到，FP16 自动混合精度对层归一化和前馈层采取了不同处理。层归一化中哪些部分对混合精度敏感？如果用 BF16 代替 FP16，是否仍需对层归一化作特殊处理？为什么？

提交内容：用 2-3 句话回答。

(c) 修改基准测试脚本，加入使用 BF16 混合精度运行模型的选项。对第 2.1.2 节各规模语言模型，分别测量启用和不启用混合精度时的前向与反向耗时。比较全精度和混合精度结果，并说明随模型规模变化所观察到的趋势。空操作上下文管理器 nullcontext 可能有所帮助。

提交内容：用 2-3 句话报告耗时并作点评。

2.1.6 内存分析

此前关注的是计算性能，现在转向语言模型训练与推理中的另一项主要资源：内存。PyTorch 提供了功能强大的内存分析器，可以随时间跟踪内存分配。

可按以下方式修改基准测试脚本：

```
... # warm-up phase in your benchmarking script

# Start recording memory history.
torch.cuda.memory._record_memory_history(max_entries=1000000)

... # what you want to profile in your benchmarking script

# Save a pickle file to be loaded by PyTorch's online tool.
torch.cuda.memory._dump_snapshot("memory_snapshot.pickle")

# Stop recording history.
torch.cuda.memory._record_memory_history(enabled=None)
```

原文第 9 页 · 原式 / 代码 / 图示

这会生成 memory_snapshot.pickle 文件，可以载入 https://pytorch.org/memory_viz 。该工具展示整体内存使用时间线，以及每次分配的大小和指向分配位置的调用栈。在浏览器中打开此链接，把 pickle 文件拖入页面即可。

现在使用 PyTorch 分析器研究模型的内存使用。

题目 memory_profiling：内存分析（4 分）

对表 1 中的 xl 模型，在上下文长度 128 和 2048 下，分析包含前向、反向和优化器更新的完整训练步骤。

(a) 在分析脚本中加入启用内存分析器的选项。可以复用之前的设施，例如启用混合精度、加载指定规模模型等。分别运行 xl 模型的仅推理（仅前向）和完整训练步骤，获取内存分析结果。内存时间线是什么样的？能否根据峰值判断正在执行哪个阶段？

提交内容：memory_viz 工具中 xl 模型的两张 Active memory timeline 图，一张为前向传播，另一张为完整训练步骤（前向、反向和优化器更新），并用 2-3 句话回答。

(b) 每种上下文长度下，前向传播的内存峰值是多少？完整训练步骤呢？

提交内容：一张表，每种上下文长度对应两个数值。

(c) 使用混合精度时，找出 xl 模型前向传播和完整训练步骤的内存峰值。混合精度是否显著影响内存使用？

提交内容：用 2-3 句话回答。

(d) 对 xl 模型，使用参考超参数时，Transformer 残差流（residual stream）中的一个激活张量，以单精度存储会占多少空间？用 MiB 表示，即将字节数除以 1024^2。

提交内容：用 1-2 句话回答，并给出推导。

(e) 仔细查看 xl 模型前向传播的 memory_viz Active Memory Timeline。降低 Detail 级别会隐藏对应比例的小额分配，例如 Detail 为 10% 时只显示最大的 10% 分配。显示出的最大分配有多大？查看调用栈，能否判断这些分配来自哪里？

提交内容：用 1-2 句话回答。

(f) Nsight Systems 也提供内存分析选项。结合前面使用的选项，了解模型运行各阶段发生了哪些分配。使用 PyTorch 提供的 NVTX 标签，确定模型中单个 TransformerBlock 为反向传播保存了多少内存；这些被保存的张量通常称为 residuals。列出贡献最大的 5 项操作，以及它们各自占总内存的百分比。

反向传播中，这些张量会被释放，但同时会产生新的梯度张量。根据前向阶段的内存分配量，以及反向阶段每经过一个 TransformerBlock 时内存使用的变化，计算一个 TransformerBlock 产生的梯度张量所占内存。结果是否符合预期？

提交内容：Nsight Systems 截图，以及 1-2 段回答。

3 单 GPU 内存

后续部分会探索如何将张量分片到多个 GPU，但也有一些技术适用于单 GPU 训练。其中最常见的就是梯度检查点（gradient checkpointing），也称为激活检查点（activation checkpointing）。

3.1 自动求导保存的中间张量

回顾一下，要在模型中执行反向传播，需要保存前向传播产生的激活。对某些操作而言，这一点很直观；但默认情况下，需要保存张量的操作可能远比你想象的多。为反向传播保存的张量称为 residuals，或者简称 saved tensors。

译注：本节的 residuals 指“为反向传播保存的中间张量”，不等同于残差连接或上一节的 residual stream。

先了解网络中究竟保存了什么。以看似简单的 RMSNorm 为例，为简化讨论，全部使用 FP32。我们添加一些钩子，在 autograd 保存或取回张量时触发：

```python
import torch
from torch import nn

x = torch.randn((4, 512, 2560), requires_grad=True)


class RMSNorm(nn.Module):
    def __init__(
        self,
        hidden_size: int,
        eps: float = 1e-5,
        device=None,
    ):
        super().__init__()
        self.weight = nn.Parameter(torch.ones(hidden_size, device=device))
        self.eps = eps

    def forward(self, x):
        rms = torch.rsqrt(x.pow(2).mean(-1, keepdim=True) + self.eps)
        x = x * rms
        return self.weight * x


def pack_hook(t):
    shape, dtype, grad_fn = t.shape, t.dtype, t.grad_fn
    print(f"Saving residual: {shape=}, {dtype=}, {grad_fn=}")
    return t


def unpack_hook(t):
    shape, dtype, grad_fn = t.shape, t.dtype, t.grad_fn
    print(f"Loading residual: {shape=}, {dtype=}, {grad_fn=}")
    return t

ln = RMSNorm(x.shape[-1])

with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = ln(x)
    y.sum().backward()
```

原文第 11 页 · 原式 / 代码 / 图示

输出显示，保存的张量多得令人担忧，其中好几个与完整激活张量一样大！

```
$ uv run scripts/autograd_experiment.py
Saving residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=<RsqrtBackward0
object at 0x7f7dd319b5e0>
Saving residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=<RsqrtBackward0
object at 0x7f7dd319b5e0>
Saving residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=<MulBackward0
object at 0x7f7dd319b5e0>
Saving residual: shape=torch.Size([2560]), dtype=torch.float32, grad_fn=None

Loading residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32,
grad_fn=<MulBackward0 object at 0x7f7cf14e6740>
```

原文第 11 页 · 原式 / 代码 / 图示

```
Loading residual: shape=torch.Size([2560]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=<RsqrtBackward0
object at 0x7f7cf14e6740>
Loading residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=<RsqrtBackward0
object at 0x7f7cf14e6740>
Loading residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
```

原文第 12 页 · 原式 / 代码 / 图示

3.1.1 算子融合

这里很明显：操作被拆得过细。我们希望用一个算子接收 RMSNorm 权重与激活并输出结果，在反向传播中也把它作为一个整体处理。这正是内核融合的动机之一。RMSNorm 的计算形式比较规整，因此甚至可以用 torch.compile 自动融合。

```
...
ln = torch.compile(RMSNorm(x.shape[-1]))

with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = ln(x)
    y.sum().backward()
```

原文第 12 页 · 原式 / 代码 / 图示

新的输出明显好得多：

```
Saving residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([4, 512, 2560]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([2560]), dtype=torch.float32, grad_fn=None
Loading residual: shape=torch.Size([4, 512, 1]), dtype=torch.float32, grad_fn=None
```

原文第 12 页 · 原式 / 代码 / 图示

现在只需为反向传播保存一个完整大小的激活张量，即 RMSNorm 的输入。还要注意，取回张量的顺序不再是保存顺序的逆序，而且每个被保存的张量都不再带有 grad_fn 依赖：PyTorch 正把整个 RMSNorm 当作一个函数处理。

3.2 激活检查点

融合无疑很有用，但它节约内存的能力有限。例如，对 xl 规模的单个 TransformerBlock 进行融合：

```
import torch
from cs336_basics.model import RotaryEmbedding, TransformerBlock

# num_layers for this model is 32
d_model, d_ff, num_heads, context_length = 2560, 10240, 16, 2048
block = TransformerBlock(d_model=d_model, d_ff=d_ff, num_heads=num_heads,
positional_encoder=RotaryEmbedding(dim=d_model // num_heads, context_length=context_length))

# Fuse as much torch.compile will allow
block = torch.compile(block, fullgraph=True)
x = torch.randn((4, context_length, d_model), requires_grad=True)
```

原文第 12 页 · 原式 / 代码 / 图示

```
...
# Now logs the number of bytes saved
total_size_bytes = 0
def pack_hook(t):
    if isinstance(t, torch.nn.Parameter):  # Skip logging parameters to avoid double counting
        return t
    global total_size_bytes
    shape, dtype, grad_fn = t.shape, t.dtype, t.grad_fn
    total_size_bytes += t.numel() * t.element_size()
    print(f"Saving residual: {shape=}, {dtype=}, {grad_fn=}")
    return t

...

# Run forward pass, saving for backward
with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = block(x)

print(f"Total size of saved tensors in single TransformerBlock: {total_size_bytes /
(1024**2):.2f} MiB")
```

原文第 13 页 · 原式 / 代码 / 图示

脚本显示了为反向传播保存的内存量：

```
...
Total size of saved tensors in single TransformerBlock: 3651.31 MiB
```

原文第 13 页 · 原式 / 代码 / 图示

每层需要约 3.6 GiB。所有层合计，仅为反向传播保存的激活就达到 114 GiB！注意力操作保存的中间张量存在相当多的冗余，我们会在第 4 节解决，但即使解决后，内存使用仍会随批大小、序列长度和嵌入维度线性增长。

译注：此处示例代码使用 num_heads=16，而表 1 的 xl 配置为 num_heads=32；两处均按原文保留。

3.2.1 重计算

不必保留产生的每个张量，我们可以只定期保存结果作为检查点，再重新计算检查点之间的值。PyTorch 提供了方便的接口 torch.utils.checkpoint.checkpoint，接收函数和该函数的参数，并按以下方式改变执行行为：

1. 前向传播：保存函数输入，并抑制函数内部为反向传播保存张量的行为。

2. 反向传播：先根据之前保存的输入重新执行前向传播，这次保存反向所需的张量；随后执行反向传播，完成后即可释放这些张量。

在顺序执行 4 个 Transformer block 的简单例子中，内存使用会按预期累加：

```
...
def four_blocks(x):
    x = block(x)
```

原文第 13 页 · 原式 / 代码 / 图示

```
    x = block(x)
    x = block(x)
    x = block(x)
    return x


with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = four_blocks(x)

print(f"Total size of saved tensors in four TransformerBlocks: {total_size_bytes /
(1024**2):.2f} MiB")
```

原文第 14 页 · 原式 / 代码 / 图示

```
Total size of saved tensors in four TransformerBlocks: 14605.25 MiB
```

原文第 14 页 · 原式 / 代码 / 图示

我们可以改为使用梯度检查点：

```
from torch.utils.checkpoint import checkpoint
def two_blocks(x):
    x = block(x)
    x = block(x)
    return x


def four_blocks_checkpoint(x):
    # checkpoint throws out all the saved tensors until the backward pass
    # when getting to the checkpointed block in the backward pass,
    # it reruns a forward pass to produce the saved tensors,
    # then completes normal backward pass.
    x = checkpoint(two_blocks, x, use_reentrant=False)
    x = checkpoint(two_blocks, x, use_reentrant=False)
    return x

with torch.autograd.graph.saved_tensors_hooks(pack_hook, unpack_hook):
    y = four_blocks_checkpoint(x)

print(f"Total size of saved tensors in four TransformerBlocks with checkpointing:
{total_size_bytes / (1024**2):.2f} MiB")
```

原文第 14 页 · 原式 / 代码 / 图示

```
Saving residual: shape=torch.Size([0]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 2048, 2560]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([0]), dtype=torch.float32, grad_fn=None
Saving residual: shape=torch.Size([4, 2048, 2560]), dtype=torch.float32,
grad_fn=<torch.autograd.function.CompiledFunctionBackward object at 0x7aa0657a19d0>
Total size of saved tensors in four TransformerBlocks with checkpointing: 160.00 MiB
```

原文第 14 页 · 原式 / 代码 / 图示

请注意，这并没有消除内存开销，而是把它分成两类：一类是在每次 checkpoint 调用入口保存、供以后重计算使用的长期存储，即检查点本身；另一类是在检查点区域内部重计算时产生、用于完成该区域反向传播的短期内存。

我们主要关心峰值内存，因此需要平衡保存检查点的成本与一次完整重建某个区域内所有中间张量的成本。检查点越多，单个区域内需要重建的内存就越少，但检查点自身占用的内存越多。

在上述例子中，重计算产生的中间张量显然占据主导，部分原因是检查点本身很小，因此适合把更多内存预算分配给检查点。也就是说，应缩小每个检查点区域覆盖的范围。

单纯改变检查点区域大小，不会改变重计算的计算量；但如果愿意用更多计算换取更少内存，可以使用递归检查点，即在 checkpoint 调用中嵌套其他 checkpoint 调用。

题目 gradient_checkpointing：内存最优的梯度检查点（4 分）

考虑由 N 个相同 block 顺序堆叠而成的 Transformer。不使用检查点时，全部 N 个 block 为反向传播保存的中间张量会同时存在，激活内存峰值为 O(N)。可以自由地对前向传播的任意部分使用 checkpoint，也可以嵌套 checkpoint。

(a) 不考虑计算成本，什么检查点策略能使激活内存峰值最小？描述 checkpoint 调用如何安排，可用代码草图说明，并给出内存峰值和计算量关于 N 的渐近复杂度。假设单个 block 保存的中间张量开销远大于每个检查点的簿记开销。

提交内容：用 3-5 句话描述策略及其渐近内存峰值，再给出一小段代码草图。

(b) 考虑上面的 xl 配置，批大小为 4、序列长度为 2048。如果时间或计算预算只允许一层重计算，即不能嵌套 checkpoint，怎样安排检查点能最好地降低峰值内存？通过性能分析测量峰值，验证你的假设；同时比较相邻的更小和更大的检查点 block 大小，确认判断。

提交内容：用 3-5 句话说明推理，并报告该策略实测的峰值内存。

4 GPU 内核

4.1 使用 FlashAttention-2 优化注意力

4.1.1 PyTorch 注意力基准测试

前面的性能分析很可能表明，注意力层在内存和计算方面都有优化空间。从整体上看，注意力计算由矩阵乘法、softmax 和另一次矩阵乘法组成：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\text{mask}\left(\frac{QK^T}{\sqrt{d_k}}\right)\right)V \tag{1}$$

原文第 15 页 · 原式 / 代码 / 图示

朴素实现需要为每个批次元素、每个注意力头保存形状为 seq_len × seq_len 的注意力分数矩阵。序列很长时，这些矩阵会变得非常大，使长输入或长输出任务发生内存不足。我们将按照 FlashAttention-2 论文实现注意力内核，以分块方式计算，避免显式生成完整的 seq_len × seq_len 注意力分数矩阵，从而支持长得多的序列。

题目 pytorch_attention：PyTorch 注意力基准测试（2 分）

(a) 测试注意力实现在不同规模下的表现。编写脚本：

(i) 将批大小固定为 8，不使用多头注意力，即去掉 head 维度。

(ii) 遍历头嵌入维度 d_model∈[16, 32, 64, 128] 与序列长度∈[256, 1024, 4096, 8192, 16384] 的笛卡尔积。

(iii) 按对应大小生成随机输入 Q、K、V。

(iv) 使用这些输入，测量 100 次注意力前向传播。

(v) 测量反向传播开始前的内存使用量，并测量 100 次反向传播。

(vi) 确保预热，并在每次前向或反向传播后调用 torch.cuda.synchronize()。

根据 GPU 不同，某些配置预计会发生内存不足。报告各配置的耗时，或者记录其内存不足错误。多大规模开始出现内存不足？从发生内存不足的最小配置中选取一个，核算注意力的内存使用量，可使用作业 1 中的 Transformer 内存公式。为反向传播保存的内存如何随序列长度变化？你会如何消除这项内存开销？

提交内容：耗时表、内存使用计算过程，以及 1-2 段回答。

4.2 即时编译注意力的基准测试

从 2.0 版开始，PyTorch 提供了强大的即时编译器，可自动尝试优化 PyTorch 函数。入门教程：https://pytorch.org/tutorials/intermediate/torch_compile_tutorial.html 。尤其是，它会动态分析计算图，尝试自动生成融合的 Triton 内核。

使用接口很简单。例如，要编译模型中的某一层：

```
layer = SomePyTorchModule(...)
compiled_layer = torch.compile(layer)
```

原文第 16 页 · 原式 / 代码 / 图示

编译后的 compiled_layer 在功能上与 layer 相同，包括前向和反向传播。也可以用 torch.compile(model) 编译整个模型，甚至编译调用 PyTorch 操作的普通 Python 函数。

题目 torch_compile：Torch Compile（2 分）

(a) 扩展注意力基准测试脚本，加入经编译的 PyTorch 注意力实现，使用与前面 pytorch_attention 相同的配置，对比编译与未编译版本的性能。

提交内容：一张表，对比编译后注意力模块与前面未编译版本的前向和反向耗时。

(b) 在端到端基准测试脚本中编译整个 Transformer 模型。前向传播性能如何变化？前向、反向和优化器更新合起来的性能呢？

提交内容：一张表，对比原始 Transformer 与编译版本。

根据已经观察到的序列长度扩展规律，要处理长序列，需要显著改进。即使用 torch.compile，当前实现面对长序列时，内存访问模式仍很差。因此，我们将用 Triton 实现 FlashAttention-2，以更精细地控制内存访问方式和计算时机。

4.2.1 示例：加权求和

为了介绍编写 Triton 所需的知识，以及 Triton 如何与 PyTorch 协作，我们先实现一个“加权求和”内核。更多学习资源可参考 Triton 教程：https://triton-lang.org/main/getting-started/tutorials/index.html 。原文指出，这些教程没有使用新的、方便的块指针（block pointer）抽象，下面将介绍这种抽象。

给定输入矩阵 X，用列方向的权重向量 w 逐元素加权，再对每一行求和，得到 X 与 w 的矩阵向量积。先实现前向，再编写反向的 Triton 内核。

前向传播

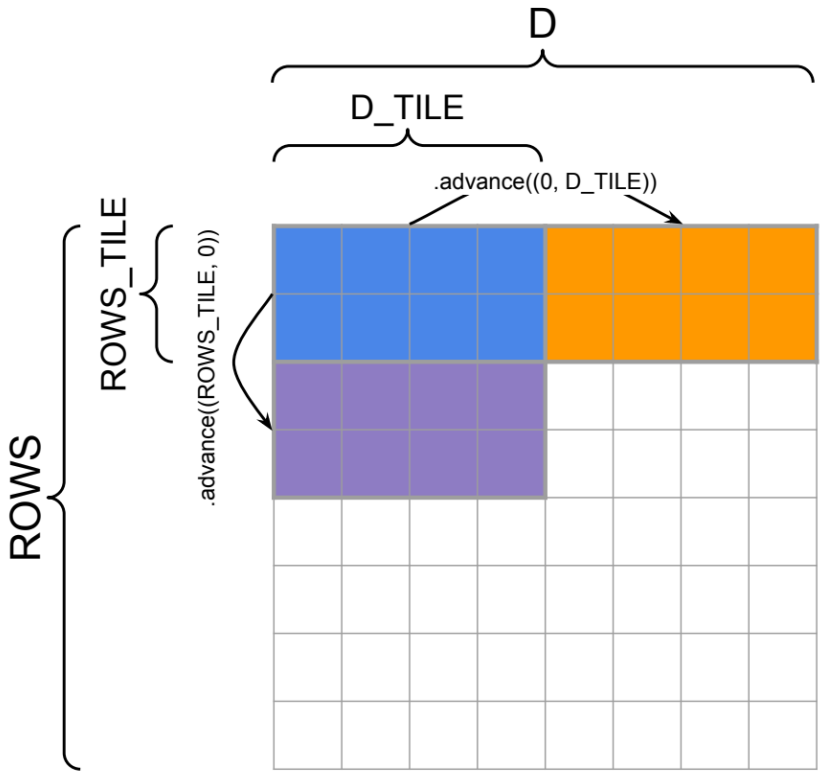
前向就是下面这个使用广播的内积运算：

```
def weighted_sum(x, weight):
    # Here, assume that x has n-dim shape [..., D], and weight has 1D shape [D]
    return (weight * x).sum(axis=-1)
```

原文第 17 页 · 原式 / 代码 / 图示

编写 Triton 内核时，让每个程序实例负责 x 的一个行分块的加权求和，并将每行对应的标量结果写入输出张量。不同程序实例可以并行执行。在 Triton 中，一个程序实例对应执行同一程序的一个线程块，这些线程块可以在 GPU 上并行运行。内核参数不是张量对象本身，而是张量首元素的指针，以及说明如何沿各轴移动的步幅（stride）。

利用步幅和程序 ID，可以加载当前实例负责的行分块：实例 i 处理 x 的第 i 个行分块。在这个简单例子中，Triton 与 PyTorch 前向实现的主要区别是，需要进行指针运算，并显式加载和存储数据。我们将用 tl.make_block_ptr 提供的块指针抽象大幅简化指针运算，不过需要先配置块指针。



原文第 18 页 · 原式 / 代码 / 图示

图 2：加权求和内核中的分块与块指针移动，见第 4.2.1 节。ROWS 表示总行数，D 表示列维度；沿行或列推进指针时，分别按对应分块大小移动。

加权求和函数的 Triton 实现如下：

```python
import triton
import triton.language as tl

@triton.jit
def weighted_sum_fwd(
    x_ptr, weight_ptr,  # Input pointers
    output_ptr,  # Output pointer
    x_stride_row, x_stride_dim,  # Strides tell us how to move one element in each axis of a tensor
    weight_stride_dim,  # Likely 1
    output_stride_row,  # Likely 1
    NUM_ROWS, D,
    ROWS_TILE_SIZE: tl.constexpr, D_TILE_SIZE: tl.constexpr,  # Tile shapes must be known at compile time
):
    # Each instance will compute the weighted sum of a tile of rows of x.
    # `tl.program_id` gives us a way to check which thread block we're running in
    row_tile_idx = tl.program_id(0)

    # Block pointers give us a way to select from an ND region of memory
    #  and move our selection around.
    # The block pointer must know:
    # - The pointer to the first element of the tensor
    # - The overall shape of the tensor to handle out-of-bounds access
    # - The strides of each dimension to use the memory layout properly
    # - The ND coordinates of the starting block, i.e., "offsets"
    # - The block shape to load/store at a time
    # - The order of the dimensions in memory from major to minor
    #    axes (= np.argsort(strides)) for optimizations, needed for
    #    TMA support on >=Hopper
```

原文第 18 页 · 原式 / 代码 / 图示

```
    x_block_ptr = tl.make_block_ptr(
        x_ptr,
        shape=(NUM_ROWS, D,),
        strides=(x_stride_row, x_stride_dim),
        offsets=(row_tile_idx * ROWS_TILE_SIZE, 0),
        block_shape=(ROWS_TILE_SIZE, D_TILE_SIZE),
        order=(1, 0),
    )

    weight_block_ptr = tl.make_block_ptr(
        weight_ptr,
        shape=(D,),
        strides=(weight_stride_dim,),
        offsets=(0,),
        block_shape=(D_TILE_SIZE,),
        order=(0,),
    )

    output_block_ptr = tl.make_block_ptr(
        output_ptr,
        shape=(NUM_ROWS,),
        strides=(output_stride_row,),
        offsets=(row_tile_idx * ROWS_TILE_SIZE,),
        block_shape=(ROWS_TILE_SIZE,),
        order=(0,),
    )

    # Initialize a buffer to write to
    output = tl.zeros((ROWS_TILE_SIZE,), dtype=tl.float32)

    for i in range(tl.cdiv(D, D_TILE_SIZE)):
        # Load the current block pointer
        # Since ROWS_TILE_SIZE might not divide NUM_ROWS, and D_TILE_SIZE might not divide D,
        # we need boundary checks for both dimensions
        row = tl.load(x_block_ptr, boundary_check=(0, 1), padding_option="zero")  # (ROWS_TILE_SIZE, D_TILE_SIZE)
        weight = tl.load(weight_block_ptr, boundary_check=(0,), padding_option="zero")  # (D_TILE_SIZE,)

        # Compute the weighted sum of the row.
        output += tl.sum(row * weight[None, :], axis=1)

        # Move the pointers to the next tile.
        # These are (rows, columns) coordinate deltas
        x_block_ptr = x_block_ptr.advance((0, D_TILE_SIZE))  # Move by D_TILE_SIZE in the last dimension
        weight_block_ptr = weight_block_ptr.advance((D_TILE_SIZE,))  # Move by D_TILE_SIZE

    # Write output to the output block pointer (a single scalar per row).
    # Since ROWS_TILE_SIZE might not divide NUM_ROWS, we need boundary checks
    tl.store(output_block_ptr, output, boundary_check=(0,))
```

原文第 19 页 · 原式 / 代码 / 图示

代码注释说明：分块形状必须在编译时已知。块指针需要知道张量首地址、完整形状、各维步幅、起始偏移、单次加载或存储的块形状，以及与内存布局相关的维度顺序；该顺序也用于 Hopper 及更新架构的 TMA 支持。若行数或 D 不能被分块大小整除，加载时需要边界检查并补零，存储时也需检查行边界。循环沿 D 维推进，累加每个行分块的加权和。

现在用 PyTorch 的 autograd.Function 包装内核，使它接收 Tensor、返回 Tensor，并能在之后接入反向传播的自动求导引擎：

```
class WeightedSumFunc(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, weight):
        # Cache x and weight to be used in the backward pass, when we
        # only receive the gradient wrt. the output tensor, and
        # need to compute the gradients wrt. x and weight.
        D, output_dims = x.shape[-1], x.shape[:-1]

        # Reshape input tensor to 2D
        input_shape = x.shape
        x = rearrange(x, "... d -> (...) d")

        ctx.save_for_backward(x, weight)
```

原文第 19 页 · 原式 / 代码 / 图示

```
        assert len(weight.shape) == 1 and weight.shape[0] == D, "Dimension mismatch"
        assert x.is_cuda and weight.is_cuda, "Expected CUDA tensors"
        assert x.is_contiguous(), "Our pointer arithmetic will assume contiguous x"

        ctx.D_TILE_SIZE = triton.next_power_of_2(D) // 16  # Roughly 16 loops through the embedding dimension
        ctx.ROWS_TILE_SIZE = 16  # Each thread processes 16 batch elements at a time
        ctx.input_shape = input_shape

        # Need to initialize empty result tensor. Note that these elements are not necessarily 0!
        y = torch.empty(output_dims, device=x.device)

        # Launch our kernel with n instances in our 1D grid.
        n_rows = y.numel()
        weighted_sum_fwd[(triton.cdiv(n_rows, ctx.ROWS_TILE_SIZE),)](
            x, weight,
            y,
            x.stride(0), x.stride(1),
            weight.stride(0),
            y.stride(0),
            NUM_ROWS=n_rows, D=D,
            ROWS_TILE_SIZE=ctx.ROWS_TILE_SIZE, D_TILE_SIZE=ctx.D_TILE_SIZE,
        )

        return y.view(input_shape[:-1])
```

原文第 20 页 · 原式 / 代码 / 图示

这里先将输入展平成二维并保存 x、weight 供反向使用，同时保留原始形状。输出用 torch.empty 分配，其内容并不保证为零。

调用 weighted_sum_fwd[(triton.cdiv(n_rows, ctx.ROWS_TILE_SIZE),)] 时，通过元组 (triton.cdiv(n_rows, ctx.ROWS_TILE_SIZE),) 指定线程块的启动网格（launch grid）。内核中可通过 tl.program_id(0) 读取线程块索引。

反向传播

既然定义了自定义内核，也需要编写自己的反向函数。前向时，给定层输入并计算输出；反向时，给定目标函数对层输出的梯度，需要计算它对各输入的梯度。

这里输入是矩阵 x∈R^(n×h) 和权重向量 w∈R^h。将该操作记为 f(x,w)，其输出空间为 R^n。已知损失 L 对层输出的梯度后，应用多元链式法则，可得到对 x 和 w 的梯度：

$$(\nabla_x \mathcal{L})_{ij} = \sum_{k=1}^{n} \frac{\partial f(x,w)_k}{\partial x_{ij}} \left(\nabla_{f(x,w)} \mathcal{L}\right)_k = w_j \cdot \left(\nabla_{f(x,w)} \mathcal{L}\right)_i \tag{2}$$

$$(\nabla_w \mathcal{L})_j = \sum_{i=1}^{n} \frac{\partial f(x,w)_i}{\partial w_j} \left(\nabla_{f(x,w)} \mathcal{L}\right)_i = \sum_{i=1}^{n} x_{ij} \cdot \left(\nabla_{f(x,w)} \mathcal{L}\right)_i \tag{3}$$

原文第 20 页 · 原式 / 代码 / 图示

式 (2) 给出 x 的梯度，本质上是输出梯度与 w 的外积。式 (3) 给出 w 的梯度：将各行输入按对应的输出梯度加权，再沿行维求和。

译注：上句按式 (3) 明确了梯度方向；原文紧随公式的文字对 input gradient 与 output row 的说法不够严谨，计算应以公式为准。

反向内核先定义所有块指针，再计算对 x 的梯度：

```
@triton.jit
def weighted_sum_backward(
    x_ptr, weight_ptr,  # Input
    grad_output_ptr,  # Grad input
    grad_x_ptr, partial_grad_weight_ptr,  # Grad outputs
    stride_xr, stride_xd,
```

原文第 20 页 · 原式 / 代码 / 图示

```
    stride_wd,
    stride_gr,
    stride_gxr, stride_gxd,
    stride_gwb, stride_gwd,
    NUM_ROWS, D,
    ROWS_TILE_SIZE: tl.constexpr, D_TILE_SIZE: tl.constexpr,
):
    row_tile_idx = tl.program_id(0)
    n_row_tiles = tl.num_programs(0)

    # Inputs
    grad_output_block_ptr = tl.make_block_ptr(
        grad_output_ptr,
        shape=(NUM_ROWS,), strides=(stride_gr,),
        offsets=(row_tile_idx * ROWS_TILE_SIZE,),
        block_shape=(ROWS_TILE_SIZE,),
        order=(0,),
    )

    x_block_ptr = tl.make_block_ptr(
        x_ptr,
        shape=(NUM_ROWS, D,), strides=(stride_xr, stride_xd),
        offsets=(row_tile_idx * ROWS_TILE_SIZE, 0),
        block_shape=(ROWS_TILE_SIZE, D_TILE_SIZE),
        order=(1, 0),
    )

    weight_block_ptr = tl.make_block_ptr(
        weight_ptr,
        shape=(D,), strides=(stride_wd,),
        offsets=(0,), block_shape=(D_TILE_SIZE,),
        order=(0,),
    )

    grad_x_block_ptr = tl.make_block_ptr(
        grad_x_ptr,
        shape=(NUM_ROWS, D,), strides=(stride_gxr, stride_gxd),
        offsets=(row_tile_idx * ROWS_TILE_SIZE, 0),
        block_shape=(ROWS_TILE_SIZE, D_TILE_SIZE),
        order=(1, 0),
    )

    partial_grad_weight_block_ptr = tl.make_block_ptr(
        partial_grad_weight_ptr,
        shape=(n_row_tiles, D,), strides=(stride_gwb, stride_gwd),
        offsets=(row_tile_idx, 0),
        block_shape=(1, D_TILE_SIZE),
        order=(1, 0),
    )

    for i in range(tl.cdiv(D, D_TILE_SIZE)):
        grad_output = tl.load(grad_output_block_ptr, boundary_check=(0,), padding_option="zero")  #
(ROWS_TILE_SIZE,)

        # Outer product for grad_x
        weight = tl.load(weight_block_ptr, boundary_check=(0,), padding_option="zero")  # (D_TILE_SIZE,)
        grad_x_row = grad_output[:, None] * weight[None, :]
        tl.store(grad_x_block_ptr, grad_x_row, boundary_check=(0, 1))

        # Reduce as many rows as possible for the grad_weight result
        row = tl.load(x_block_ptr, boundary_check=(0, 1), padding_option="zero")  # (ROWS_TILE_SIZE, D_TILE_SIZE)
        grad_weight_row = tl.sum(row * grad_output[:, None], axis=0, keep_dims=True)
        tl.store(partial_grad_weight_block_ptr, grad_weight_row, boundary_check=(1,))  # Never out of bounds for dim
0


        # Move the pointers to the next tile along D
        x_block_ptr = x_block_ptr.advance((0, D_TILE_SIZE))
        weight_block_ptr = weight_block_ptr.advance((D_TILE_SIZE,))
        partial_grad_weight_block_ptr = partial_grad_weight_block_ptr.advance((0, D_TILE_SIZE))
        grad_x_block_ptr = grad_x_block_ptr.advance((0, D_TILE_SIZE))
```

原文第 21 页 · 原式 / 代码 / 图示

计算 x 的梯度很简单，直接写到输出张量对应的分块即可。计算 w 的梯度则更麻烦：每个内核实例只负责 x 的一个行分块，但这里需要对所有行求和。

我们不在反向内核中直接完成整个求和，而是假设 partial_grad_weight_ptr 指向 n_row_tiles × H 的矩阵，每个条目仅包含对应行分块内部的归约结果。先在当前行分块内归约，再写入该张量；随后在内核外使用 torch.sum 汇总各个行分块，得到 w 的最终梯度。[脚注 1：当然，也可以为这一步自行编写内核。]

autograd.Function 剩下的部分就相对简单了：

```
class WeightedSumFunc(torch.autograd.Function):
    @staticmethod
    def forward(ctx, x, weight):
        # ... (defined earlier)

    @staticmethod
    def backward(ctx, grad_out):
        x, weight = ctx.saved_tensors
        ROWS_TILE_SIZE, D_TILE_SIZE = ctx.ROWS_TILE_SIZE, ctx.D_TILE_SIZE  # These don't have to be the same
        n_rows, D = x.shape

        # Our strategy is for each thread block to first write to a partial buffer,
        # then we reduce over this buffer to get the final gradient.
        partial_grad_weight = torch.empty((triton.cdiv(n_rows, ROWS_TILE_SIZE), D), device=x.device, dtype=x.dtype)
        grad_x = torch.empty_like(x)

        weighted_sum_backward[(triton.cdiv(n_rows, ROWS_TILE_SIZE),)](
            x, weight,
            grad_out,
            grad_x, partial_grad_weight,
            x.stride(0), x.stride(1),
            weight.stride(0),
            grad_out.stride(0),
            grad_x.stride(0), grad_x.stride(1),
            partial_grad_weight.stride(0), partial_grad_weight.stride(1),
            NUM_ROWS=n_rows, D=D,
            ROWS_TILE_SIZE=ROWS_TILE_SIZE, D_TILE_SIZE=D_TILE_SIZE,
        )
        grad_weight = partial_grad_weight.sum(axis=0)
        return grad_x, grad_weight
```

原文第 22 页 · 原式 / 代码 / 图示

最后，通过下面的方式得到一个用起来类似 torch.nn.functional 中函数的接口：

```
f_weightedsum = WeightedSumFunc.apply
```

原文第 22 页 · 原式 / 代码 / 图示

在两个 PyTorch 张量 x 和 w 上调用 f_weightedsum，会得到如下张量：

```
tensor([ 90.8563, -93.6815, -80.8884,  ..., 103.4840, -21.4634, -24.0192],
       device='cuda:0', grad_fn=<WeightedSumFuncBackward>)
```

原文第 22 页 · 原式 / 代码 / 图示

注意输出附带的 grad_fn：这表明，当该张量出现在计算图中时，PyTorch 知道反向传播应该调用哪个函数。至此，加权求和的 Triton 实现完成。

4.2.2 FlashAttention-2 前向传播

接下来将依据 FlashAttention-2 [T. Dao, 2023]，用性能显著更好的 Triton 实现替换 PyTorch 注意力。FlashAttention-2 通过一些技巧按分块计算前向传播，从而实现高效的内存访问，并避免在全局内存中显式生成完整注意力矩阵。

开始前，强烈建议至少阅读最初的 FlashAttention 论文 [T. Dao 等，2022]，理解其核心技术：跨分块在线计算 softmax，这一技术由 M. Milakov 等在 2018 年提出。也建议阅读 H. He 的资料 [4]，建立 GPU 实际如何执行 PyTorch 代码的直觉。

理解普通注意力的低效之处

暂时忽略掩码，注意力前向传播可写为：

$$\boldsymbol{S} = \boldsymbol{Q}\boldsymbol{K}^\top / \sqrt{d} \tag{4}$$
$$P_{ij} = \mathrm{softmax}_j(\boldsymbol{S})_{ij} \tag{5}$$
$$\boldsymbol{O} = \boldsymbol{P}\boldsymbol{V} \tag{6}$$

原文第 23 页 · 原式 / 代码 / 图示

标准反向传播为：

$$\boldsymbol{dV} = \boldsymbol{P}^\top \boldsymbol{dO} \tag{7}$$
$$\boldsymbol{dP} = \boldsymbol{dO}\boldsymbol{V}^\top \tag{8}$$
$$\boldsymbol{dS}_i = \mathrm{dsoftmax}(\boldsymbol{dP}_i) = (\mathrm{diag}(\boldsymbol{P}_i) - \boldsymbol{P}_i\boldsymbol{P}_i^\top)\boldsymbol{dP}_i \tag{9}$$
$$\boldsymbol{dQ} = \boldsymbol{dS}\boldsymbol{K} / \sqrt{d} \tag{10}$$
$$\boldsymbol{dK} = \boldsymbol{dS}^\top \boldsymbol{Q} / \sqrt{d} \tag{11}$$

原文第 23 页 · 原式 / 代码 / 图示

可以看到，反向传播依赖前向阶段非常大的激活。例如，式 (7) 中计算 dV 需要 P，其形状为 (batch_size, n_heads, seq_len, seq_len)。该激活矩阵的大小随序列长度平方增长，这解释了前面长序列注意力测试遇到的内存问题。

普通注意力在前向和反向过程中都需要在片上 SRAM 与 GPU HBM 之间搬运 P 及其他大型激活，产生显著的内存 I/O 开销。标准实现会多次搬运这些数据，例如反向传播的式 (7) 和式 (9) 都会从 HBM 读取 P。

FlashAttention 的主要目标是避免把注意力矩阵读写到 HBM，从而降低 I/O 和内存峰值。实现这一目标依靠三种技术：分块、重计算和算子融合。

分块（tiling）

为了避免在 HBM 中读写完整注意力矩阵，我们要在无法同时访问全部输入的情况下完成 softmax 归约。具体来说，重新组织注意力计算，把输入拆成分块，并多次遍历输入分块，逐步完成 softmax 归约。

重计算（recomputation）

不把形状为 (batch_size, n_heads, seq_len, seq_len) 的大型中间注意力矩阵保存在 HBM 中，而只保存某些“激活检查点”。反向传播时重算部分前向过程，以获得计算梯度所需的其他激活。FlashAttention-2 还保存注意力分数的 logsumexp，记作 L，用于简化反向计算：

$$L_i = \log\left(\sum_j \exp(\boldsymbol{S}_{ij})\right) \tag{12}$$

原文第 23 页 · 原式 / 代码 / 图示

最终内核会在线计算这个量，但结果应与该表达式一致。原文指出，结合分块与重计算后，内存 I/O 和峰值使用量不再依赖 sequence_length²，因而可以使用更长序列。

译注：这里保留原文的概括。严格来说，应区分“不显式存储平方规模的注意力矩阵”与“总 HBM I/O 的渐近复杂度”；后者还取决于分块方式及片上存储容量。

算子融合（operator fusion）

最后，将全部操作合并到一个内核中，避免注意力矩阵和其他中间激活的反复内存 I/O，这就是算子融合或内核融合。我们将编写一个 Triton 前向内核，执行注意力的全部操作，并限制 HBM 与 SRAM 之间的数据搬运。

重计算也使融合更容易，因为不再需要像普通实现那样，把每个中间激活都写入 HBM。关于这些技巧的更多直觉，可参阅两篇 FlashAttention 论文 [T. Dao 等，2022；T. Dao，2023]。

使用重计算的反向传播

利用 L，可以完成所需重计算，并高效执行反向传播。开始反向前，在全局内存中预先计算 D = rowsum(O ∘ dO)，其中 ∘ 表示逐元素乘法。该值等于 rowsum(P ∘ dP)，因为 $P \cdot dP^T = P \cdot (dO \cdot V^T)^T = (P \cdot V) \cdot dO^T = O \cdot dO^T$；对任意矩阵 A、B，还有 rowsum(A ∘ B) = diag(A·B^T)。

有了 L 和 D 两个向量，反向传播无需调用 softmax。完整计算如下：

$$\boldsymbol{S} = \boldsymbol{Q}\boldsymbol{K}^\top/\sqrt{d} \tag{13}$$

$$P_{ij} = \exp(S_{ij} - L_i) \tag{14}$$

$$\boldsymbol{dV} = \boldsymbol{P}^\top \boldsymbol{dO} \tag{15}$$

$$\boldsymbol{dP} = \boldsymbol{dO}\boldsymbol{V}^\top \tag{16}$$

$$dS_{ij} = P_{ij}(dP_{ij} - D_i) \tag{17}$$

$$\boldsymbol{dQ} = \boldsymbol{dS}\boldsymbol{K}/\sqrt{d} \tag{18}$$

$$\boldsymbol{dK} = \boldsymbol{dS}^\top \boldsymbol{Q}/\sqrt{d} \tag{19}$$

原文第 24 页 · 原式 / 代码 / 图示

这一系列操作不要求前向传播时把注意力分数 P 保存到 HBM：式 (13) 和式 (14) 会根据 Q、K、L 重新计算它。

FlashAttention 前向传播的细节

了解总体思路后，下面深入将要实现的 FA2 前向内核。为了避免把注意力矩阵读写到 HBM，希望按分块计算，各输出分块彼此独立。这要求能够分块计算 P，最好同时沿 query 和 key 两个维度分块。

但对 S 做 softmax 时，需要对 S 的整行归约以求分母，因此不能直接逐块计算 P。FlashAttention-2 使用在线 softmax 解决这一问题。下文以下标 i 表示当前 query 分块，以上标 (j) 表示当前 key 分块。query 方向的分块大小为 B_q，key 方向为 B_k，不沿隐藏维度 d 分块。

还要按行维护两个运行状态 m_i^(j) 和 l_i^(j)，它们都是长度 B_q 的向量。m_i^(j) 是截至当前的逐行最大值，用来数值稳定地计算 softmax，正如作业 1 实现 softmax 时使用的技巧。每处理一个新的 S 分块，也就是 j 增大时，都更新最大值。

基于运行最大值，计算未归一化的 softmax 值（分子）：P_i^(j) = exp(S_i^(j) - m_i^(j))。l_i^(j) 是 softmax 分母的运行累加量，按如下方式更新：l_i^(j) = exp(m_i^(j-1) - m_i^(j)) · l_i^(j-1) + rowsum(P_i^(j))。

最终写出结果前，使用处理完全部 key 分块后的最终值 l_i^(T_k) 完成归一化。算法 1 展示 GPU 上的前向计算。

Algorithm 1: FlashAttention-2 forward pass

Require: $\boldsymbol{Q} \in \mathbb{R}^{N_q \times d}$, $\boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{N_k \times d}$, tile sizes B_q, B_k
Split $\boldsymbol{Q}$ into $T_q = \left\lceil \frac{N_q}{B_q} \right\rceil$ tiles $\boldsymbol{Q}_1, \ldots, \boldsymbol{Q}_{T_q}$ of size $B_q \times d$
Split $\boldsymbol{K}, \boldsymbol{V}$ into $T_k = \left\lceil \frac{N_k}{B_k} \right\rceil$ tiles $\boldsymbol{K}^{(1)}, \ldots, \boldsymbol{K}^{(T_k)}$ and $\boldsymbol{V}^{(1)}, \ldots, \boldsymbol{V}^{(T_k)}$ of size $B_k \times d$
for $i = 1, \ldots, T_q$ **do**
  Load $\boldsymbol{Q}_i$ from global memory
  Initialize $\boldsymbol{O}_i^{(0)} = \boldsymbol{0} \in \mathbb{R}^{B_q \times d}, l_i^{(0)} = 0 \in \mathbb{R}^{B_q}, m_i^{(0)} = -\infty \in \mathbb{R}^{B_q}$
  for $j = 1, \ldots, T_k$ **do**
    Load $\boldsymbol{K}^{(j)}, \boldsymbol{V}^{(j)}$ from global memory
    Compute tile of pre-softmax attention scores $\boldsymbol{S}_i^{(j)} = \frac{\boldsymbol{Q}_i (\boldsymbol{K}^{(j)})^\top}{\sqrt{d}} \in \mathbb{R}^{B_q \times B_k}$
    Compute $m_i^{(j)} = \max\left(m_i^{(j-1)}, \text{rowmax}\left(\boldsymbol{S}_i^{(j)}\right)\right) \in \mathbb{R}^{B_q}$
    Compute $\tilde{\boldsymbol{P}}_i^{(j)} = \exp\left(\boldsymbol{S}_i^{(j)} - m_i^{(j)}\right) \in \mathbb{R}^{B_q \times B_k}$
    Compute $l_i^{(j)} = \exp\left(m_i^{(j-1)} - m_i^{(j)}\right) l_i^{(j-1)} + \text{rowsum}\left(\tilde{\boldsymbol{P}}_i^{(j)}\right) \in \mathbb{R}^{B_q}$
    Compute $\boldsymbol{O}_i^{(j)} = \text{diag}\left(\exp\left(m_i^{(j-1)} - m_i^{(j)}\right)\right) \boldsymbol{O}_i^{(j-1)} + \tilde{\boldsymbol{P}}_i^{(j)} \boldsymbol{V}^{(j)}$
  end for
  Compute $\boldsymbol{O}_i = \text{diag}\left(l_i^{(T_k)}\right)^{-1} \boldsymbol{O}_i^{(T_k)}$
  Compute $L_i = m_i^{(T_k)} + \log\left(l_i^{(T_k)}\right)$
  Write $\boldsymbol{O}_i$ to global memory as the i-th tile of $\boldsymbol{O}$.
  Write L_i to global memory as the i-th tile of L.
end for
Return the output $\boldsymbol{O}$ and the logsumexp L.

原文第 25 页 · 原式 / 代码 / 图示

算法 1：FlashAttention-2 前向传播。为保证符号准确，上方保留原算法；逐行中文说明如下。

1. 输入 Q∈R^(N_q×d)、K,V∈R^(N_k×d)，以及分块大小 B_q、B_k。

2. 将 Q 分成 T_q=⌈N_q/B_q⌉ 个大小为 B_q×d 的块。

3. 将 K、V 分别分成 T_k=⌈N_k/B_k⌉ 个大小为 B_k×d 的块。

4-6. 遍历 query 分块 i，从全局内存加载 Q_i；初始化输出累加器为零、l 为零、m 为负无穷。

7-9. 遍历 key 分块 j，加载 K^(j)、V^(j)，计算该分块的 softmax 前注意力分数。

10-11. 更新逐行最大值，并减去该最大值后取指数，得到未归一化概率 P。

12. 用新旧最大值的差对旧分母累加量重新缩放，再加上当前 P 的逐行和。

13. 以相同方式重新缩放旧输出累加器，再加上 P 与当前 V 分块的乘积。

14-16. 结束 key 循环，使用最终 l 对输出归一化，并计算 L_i = m_i^(T_k) + log(l_i^(T_k))。

17-20. 将 O_i 与 L_i 分别写回全局内存中 O 和 L 的第 i 块；结束 query 循环，返回 O 和 logsumexp 值 L。

Triton 使用提示

在实现前向内核之前，先汇总一些编写 Triton 的通用技巧。

• 可以用 tl.device_print 打印调试信息，文档：https://triton-lang.org/main/python-api/generated/triton.language.device_print.html 。设置 TRITON_INTERPRET=1 可在 CPU 上运行 Triton 解释器，不过课程组发现它存在一些问题。

• 定义块指针时，确保偏移正确，并将分块索引乘以对应分块大小。

• 与加权求和示例一样，在 torch.autograd.Function 子类的方法中，通过 kernel_fn[(launch_grid_d1, launch_grid_d2, ...)](...arguments...) 设置线程块启动网格。

• 使用 tl.dot 执行矩阵乘法。

• 使用 *_block_ptr = *_block_ptr.advance(...) 推进块指针。

题目 flash_forward：FlashAttention-2 前向传播（15 分）

(a) 使用纯 PyTorch，不用 Triton，编写实现 FlashAttention-2 前向传播的 autograd.Function。这会比普通 PyTorch 注意力实现慢很多，但有助于调试后面的 Triton 内核。

实现接收 Q、K、V 和 is_causal 标志，计算输出 O 与 logsumexp 值 L。此小问可以忽略 is_causal。autograd.Function 的 forward 应保存 L、Q、K、V、O 供反向使用，并返回 O。注意，forward 的第一个参数始终是上下文 ctx。autograd.Function 子类都需要实现 backward，但目前可以让它直接抛出 NotImplementedError。需要参考结果时，可用 PyTorch 实现式 (4)-(6) 及式 (12)，再比较输出。

接口为 def forward(ctx, Q, K, V, is_causal=False)。分块大小自行决定，但至少为 16×16。测试所用维度始终是至少为 16 的 2 的幂，因此无需担心越界访问。

提交内容：一个在前向中实现 FlashAttention-2 的 torch.autograd.Function 子类。实现 adapters.get_flashattention_autograd_function_pytorch，再运行 uv run pytest -k test_flash_forward_pass_pytorch，确保通过。

(b) 按算法 1 编写 FlashAttention-2 前向的 Triton 内核。再编写另一个 torch.autograd.Function 子类，在前向中调用这个融合内核，而不再用 PyTorch 逐步计算。提示如下：

• 调试时，建议将每个 Triton 操作的结果与 (a) 中的分块 PyTorch 实现逐项比较。

• 启动网格应为 (T_q, batch_size)，每个 Triton 实例只加载一个 batch 索引的数据，并仅对 Q、O、L 的一个 query 分块进行读写。

• 内核只应包含一个循环，遍历 key 分块 1≤j≤T_k。

• 在循环末尾推进块指针。

• 使用以下函数声明。根据提供的块指针，应能推断其他指针的配置方式。

```
@triton.jit
def flash_fwd_kernel(
```

原文第 26 页 · 原式 / 代码 / 图示

```
    Q_ptr, K_ptr, V_ptr,
    O_ptr, L_ptr,
    stride_qb, stride_qq, stride_qd,
    stride_kb, stride_kk, stride_kd,
    stride_vb, stride_vk, stride_vd,
    stride_ob, stride_oq, stride_od,
    stride_lb, stride_lq,
    N_QUERIES, N_KEYS,
    scale,
    D: tl.constexpr,
    Q_TILE_SIZE: tl.constexpr,
    K_TILE_SIZE: tl.constexpr,
):
    # Program indices
    query_tile_index = tl.program_id(0)
    batch_index = tl.program_id(1)

    # Offset each pointer with the corresponding batch index
    # multiplied with the batch stride for each tensor
    Q_block_ptr = tl.make_block_ptr(
        Q_ptr + batch_index * stride_qb,
        shape=(N_QUERIES, D),
        strides=(stride_qq, stride_qd),
        offsets=(query_tile_index * Q_TILE_SIZE, 0),
        block_shape=(Q_TILE_SIZE, D),
        order=(1, 0),
    )

    ...
```

原文第 27 页 · 原式 / 代码 / 图示

其中 scale=1/√d，Q_TILE_SIZE 和 K_TILE_SIZE 分别对应 B_q、B_k，之后可以调优。

为避免精度问题，还应注意：

• 片上缓冲区 O_i、l、m 的数据类型应为 tl.float32。如果向输出缓冲区累加，使用 acc 参数，即 acc = tl.dot(..., acc=acc)。

• 将 P_i^(j) 与 V^(j) 相乘之前，先把前者转换为后者的数据类型；写回全局内存前，也要将 O_i 转换为合适类型。使用 tensor.to 转换，通过 tensor.dtype 查询张量类型，通过 *_block_ptr.type.element_ty 查询块指针或指针所指元素的类型。

提交内容：一个使用 Triton 内核在前向中实现 FlashAttention-2 的 torch.autograd.Function 子类。实现 adapters.get_flash_autograd_function_triton，再运行 uv run pytest -k test_flash_forward_pass_triton，确保通过。

(c) 在 autograd.Function 实现的最后一个参数位置添加因果掩码标志。它是布尔值，设为 True 时，启用基于索引比较的因果掩码。Triton 内核应增加相应参数 is_causal: tl.constexpr，此类型注解是必需的。

在 Triton 中构造 query 和 key 的索引向量，通过比较生成 B_q×B_k 的掩码。对于被屏蔽的元素，向注意力分数矩阵 S_i^(j) 对应位置加上常数 -1e6。使用 ctx.is_causal = is_causal 保存标志供反向使用。

提交内容：给使用 Triton 内核的 FlashAttention-2 前向 autograd.Function 子类添加因果掩码标志。该参数应为可选参数，默认 False，确保之前的测试仍能通过。

通过重计算实现反向传播

与式 (7)-(11) 的标准反向不同，式 (13)-(19) 可以用重计算避免反向中的 softmax。因此可以使用简单内核完成，不再需要在线计算技巧。本部分允许用普通 PyTorch 函数配合 torch.compile 实现反向，无需 Triton。

题目 flash_backward：FlashAttention-2 反向传播（5 分）

使用 PyTorch（不用 Triton）和 torch.compile，为 FlashAttention-2 的 autograd.Function 实现反向传播。输入为 Q、K、V、O、dO、L，返回 dQ、dK、dV。记得计算并使用 D 向量。可以遵循式 (13)-(19)。

提交内容：运行 uv run pytest -k test_flash_backward 测试实现。

下面比较部分使用 Triton 的 FlashAttention-2 与普通 PyTorch 注意力的性能。

题目 flash_benchmarking：FlashAttention-2 基准测试（5 分）

(a) 使用 triton.testing.do_bench（文档：https://triton-lang.org/main/python-api/generated/triton.testing.do_bench.html ）编写脚本，对比部分使用 Triton 的 FlashAttention-2 前向、反向与普通 PyTorch 注意力（不使用 FlashAttention）。

报告一张表，分别列出 Triton 与 PyTorch 的前向延迟、反向延迟，以及前向加反向的端到端延迟。在计时之前随机生成所有需要的输入，并在单张 B200 上运行。批大小始终为 1，启用因果掩码。遍历以下参数的笛卡尔积：序列长度为 128 到 65536 之间的各个 2 的幂；嵌入维度为 16 到 128 之间的各个 2 的幂；精度为 torch.bfloat16 和 torch.float32。你可能需要根据输入大小调整分块大小。

提交内容：按上述设置比较 FlashAttention-2 与 PyTorch 的结果表，包含前向、反向和端到端延迟。

4.2.3 可选：使用 Triton 实现反向传播

如果希望进一步练习 Triton，或者提交更快的排行榜实现，下方提供了可用 Triton 实现的分块 FlashAttention-2 反向算法。

算法 2 展示应如何在 Triton 中实现。关键技巧是计算两次 P：一次用于 dQ，另一次用于 dK 和 dV。这样可以避免线程块间同步，也就不需要缓慢的原子操作。

Algorithm 2: Tiled FlashAttention-2 backward pass

Require: $\boldsymbol{Q}, \boldsymbol{O}, \boldsymbol{dO} \in \mathbb{R}^{N_q \times d}$, $\boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{N_k \times d}$, $L \in \mathbb{R}^{N_q}$, tile sizes B_q, B_k
Compute $D = \mathrm{rowsum}(\boldsymbol{O} \circ \boldsymbol{dO}) \in \mathbb{R}^{N_q}$
Split $\boldsymbol{Q}, \boldsymbol{O}, \boldsymbol{dO}$ into $T_q = \left\lceil \frac{N_q}{B_q} \right\rceil$ tiles $\boldsymbol{Q}_1, ..., \boldsymbol{Q}_{T_q}$, $\boldsymbol{O}_1, ..., \boldsymbol{O}_{T_q}$, $\boldsymbol{dO}_1, ..., \boldsymbol{dO}_{T_q}$, each of size $B_q \times d$
Split $\boldsymbol{K}, \boldsymbol{V}$ into $T_k = \left\lceil \frac{N_k}{B_k} \right\rceil$ tiles $\boldsymbol{K}^{(1)}, ..., \boldsymbol{K}^{(T_k)}$ and $\boldsymbol{V}^{(1)}, ..., \boldsymbol{V}^{(T_k)}$, each of size $B_k \times d$
Split L, D into T_q tiles $L_1, ..., L_{T_q}$ and $D_1, ..., D_{T_q}$, each of size B_q
for $j = 1, ..., T_k$ **do**
  Load $\boldsymbol{K}^{(j)}, \boldsymbol{V}^{(j)}$ from global memory
  Initialize $\boldsymbol{dK}_0^{(j)} = \boldsymbol{dV}_0^{(j)} = \boldsymbol{0} \in \mathbb{R}^{B_k \times d}$
  for $i = 1, ..., T_q$ **do**
    Load $\boldsymbol{Q}_i, \boldsymbol{dO}_i$ from global memory
    Compute tile of attention scores $\boldsymbol{S}_i^{(j)} = \frac{\boldsymbol{Q}_i (\boldsymbol{K}^{(j)})^\top}{\sqrt{d}} \in \mathbb{R}^{B_q \times B_k}$
    Compute attention probabilities $\boldsymbol{P}_i^{(j)} = \exp\left(\boldsymbol{S}_i^{(j)} - L_i\right) \in \mathbb{R}^{B_q \times B_k}$
    Compute $\boldsymbol{dV}_i^{(j)} = \boldsymbol{dV}_{i-1}^{(j)} + \left(\boldsymbol{P}_i^{(j)}\right)^\top \boldsymbol{dO}_i \in \mathbb{R}^{B_k \times d}$
    Compute $\boldsymbol{dP}_i^{(j)} = \boldsymbol{dO}_i \left(\boldsymbol{V}^{(j)}\right)^\top \in \mathbb{R}^{B_q \times B_k}$
    Compute $\boldsymbol{dS}_i^{(j)} = \boldsymbol{P}_i^{(j)} \circ \left(\boldsymbol{dP}_i^{(j)} - D_i\right) \in \mathbb{R}^{B_q \times B_k}$
    Compute $\boldsymbol{dK}_i^{(j)} = \boldsymbol{dK}_{i-1}^{(j)} + \left(\boldsymbol{dS}_i^{(j)}\right)^\top \boldsymbol{Q}_i / \sqrt{d} \in \mathbb{R}^{B_k \times d}$.
  end for
  Write $\boldsymbol{dK}_{T_q}^{(j)}$ and $\boldsymbol{dV}_{T_q}^{(j)}$ to global memory as the j-th tiles of $\boldsymbol{dK}$ and $\boldsymbol{dV}$.
end for
for $i = 1, ..., T_q$ **do**
  Load $\boldsymbol{Q}_i, \boldsymbol{dO}_i$ from global memory
  Initialize $\boldsymbol{dQ}_i^{(0)} = \boldsymbol{0} \in \mathbb{R}^{B_q \times d}$
  for $j = 1, ..., T_k$ **do**
    Load $\boldsymbol{K}^{(j)}, \boldsymbol{V}^{(j)}$ from global memory
    Compute tile of attention scores $\boldsymbol{S}_i^{(j)} = \frac{\boldsymbol{Q}_i (\boldsymbol{K}^{(j)})^\top}{\sqrt{d}} \in \mathbb{R}^{B_q \times B_k}$
    Compute attention probabilities $\boldsymbol{P}_i^{(j)} = \exp\left(\boldsymbol{S}_i^{(j)} - L_i\right) \in \mathbb{R}^{B_q \times B_k}$
    Compute $\boldsymbol{dP}_i^{(j)} = \boldsymbol{dO}_i \left(\boldsymbol{V}^{(j)}\right)^\top \in \mathbb{R}^{B_q \times B_k}$
    Compute $\boldsymbol{dS}_i^{(j)} = \boldsymbol{P}_i^{(j)} \circ \left(\boldsymbol{dP}_i^{(j)} - D_i\right) \in \mathbb{R}^{B_q \times B_k}$
    Compute $\boldsymbol{dQ}_i^{(j)} = \boldsymbol{dQ}_i^{(j-1)} + \boldsymbol{dS}_i^{(j)} \boldsymbol{K}^{(j)} / \sqrt{d} \in \mathbb{R}^{B_q \times d}$
  end for
  Write $\boldsymbol{dQ}_i^{(T_k)}$ to global memory as the i-th tile of $\boldsymbol{dQ}$.
end for
Return $\boldsymbol{dQ}, \boldsymbol{dK}, \boldsymbol{dV}$.

原文第 29 页 · 原式 / 代码 / 图示

算法 2：分块 FlashAttention-2 反向传播。逐行中文说明如下。

1. 输入 Q、O、dO（N_q×d），K、V（N_k×d），L（长度 N_q），以及 B_q、B_k。

2. 计算 D = rowsum(O ∘ dO)。

3-5. 将 Q、O、dO 按 B_q 行分成 T_q 块；将 K、V 按 B_k 行分成 T_k 块；将 L、D 按 B_q 个元素分成 T_q 块。

6-8. 遍历 key 分块 j，加载 K^(j)、V^(j)，将该分块的 dK 和 dV 累加器初始化为零。

9-12. 遍历 query 分块 i，加载 Q_i、dO_i；计算该分块的注意力分数 S，再利用保存的 L_i 计算概率 P = exp(S-L_i)。

13. 累加 dV：把当前 P 转置后与 dO_i 相乘，并加到已有结果上。

14-15. 计算 $dP = dO_i \cdot V^{(j)T}$，再计算 dS = P∘(dP-D_i)。

16-19. 累加 dK，完成全部 query 分块后，将最终 dK、dV 写回全局内存中相应的 key 分块；结束第一轮外循环。

20-22. 再遍历 query 分块 i，加载 Q_i、dO_i，把该分块的 dQ 累加器初始化为零。

23-28. 遍历 key 分块 j，加载 K^(j)、V^(j)，重新计算 S、P、dP 和 dS。

29-32. 按 dS·K^(j)/√d 累加 dQ；完成全部 key 分块后，将 dQ 写回全局内存中对应的 query 分块。

33. 返回 dQ、dK、dV。

5 分布式数据并行训练

接下来探索如何用多个 GPU 训练语言模型，重点是数据并行。先介绍 PyTorch 分布式通信基础，再研究朴素的分布式数据并行实现，随后实现并评估不同的通信效率优化。

5.1 PyTorch 中的单节点分布式通信

先看一个简单的 PyTorch 分布式程序：生成四个随机整数张量，并计算它们的和。

下面的分布式版本启动四个工作进程，每个进程生成一个随机整数张量。为了跨工作进程求和，调用 all-reduce 集合通信操作，将每个进程原来的数据张量替换为归约结果，即总和。代码如下：

```python
import os
import torch
import torch.distributed as dist
import torch.multiprocessing as mp

def setup(rank, world_size):
    os.environ["MASTER_ADDR"] = "localhost"
    os.environ["MASTER_PORT"] = "29500"
    dist.init_process_group("gloo", rank=rank, world_size=world_size)

def distributed_demo(rank, world_size):
    setup(rank, world_size)
    data = torch.randint(0, 10, (3,))
    print(f"rank {rank} data (before all-reduce): {data}")
    dist.all_reduce(data, async_op=False)
    print(f"rank {rank} data (after all-reduce): {data}")

if __name__ == "__main__":
    world_size = 4
    mp.spawn(fn=distributed_demo, args=(world_size, ), nprocs=world_size, join=True)
```

原文第 30 页 · 原式 / 代码 / 图示

运行后会得到以下输出。正如预期，各工作进程最初持有不同张量；跨进程求和的 all-reduce 完成后，每个进程的 data 都被原地更新为同一结果。

```
$ uv run python distributed_hello_world.py
rank 3 data (before all-reduce): tensor([3, 7, 8])
rank 0 data (before all-reduce): tensor([4, 4, 7])
rank 2 data (before all-reduce): tensor([6, 0, 7])
rank 1 data (before all-reduce): tensor([9, 5, 3])
rank 1 data (after all-reduce): tensor([22, 16, 25])
rank 0 data (after all-reduce): tensor([22, 16, 25])
rank 3 data (after all-reduce): tensor([22, 16, 25])
rank 2 data (after all-reduce): tensor([22, 16, 25])
```

原文第 30 页 · 原式 / 代码 / 图示

[脚注 2] 多次运行脚本会发现打印顺序不确定。程序在分布式环境中运行，无法控制命令的精确执行顺序；唯一保证是 all-reduce 完成后，各进程的结果张量按位相同。

仔细看脚本：mp.spawn 启动 nprocs 个进程，以给定 args 执行 fn；实际调用形式为 fn(rank, *args)，rank 是 0 到 nprocs-1 之间的工作进程索引。因此 distributed_demo 的第一个位置参数必须接收整数 rank。还传入 world_size，表示工作进程总数。

各工作进程属于一个进程组，通过 dist.init_process_group 初始化。进程组表示通过共同的主节点协调和通信的多个进程。主节点由 IP 地址和端口确定，rank 0 进程运行在主节点上。all-reduce 等集合通信操作作用于进程组中的各进程。

此处使用 gloo 后端，也可选择其他后端。特别是 nccl 后端使用 NVIDIA NCCL 集合通信库，对 CUDA 张量通常更高效。但 NCCL 只能用于有 GPU 的机器，Gloo 则可在纯 CPU 机器上运行。GPU 分布式训练应始终使用 NCCL；没有 GPU 的本地开发才使用 Gloo。示例采用 Gloo，是为了让它也能在纯 CPU 机器上执行和调试。

运行多 GPU 作业时，要确保不同 rank 使用不同 GPU。一种方法是在 setup 中调用 torch.cuda.set_device(rank)，这样 tensor.to("cuda") 会自动移到指定设备。另一种方法是为每个 rank 显式构造设备字符串，例如 device = f"cuda:{rank}"，并在移动数据时使用它，例如 tensor.to(f"cuda:{rank}")。

术语

本作业余下部分及其他资料会使用以下术语。虽然本作业聚焦单节点、多进程训练，这些术语也适用于一般分布式训练，见图 3。

• node（节点）：网络中的一台机器。

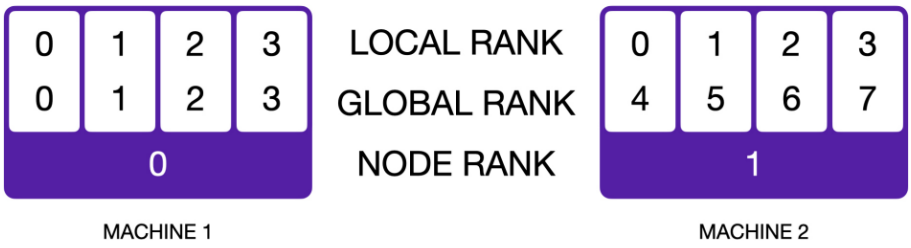
• worker（工作单元）：参与分布式训练的一个程序实例。本作业中，每个 worker 只有一个进程，因此 worker、process 和 worker process 交替使用。但实际中一个 worker 可能有多个进程，例如用于数据加载，因此这些词并非总是等价。

• world size：一个进程组中 worker 的总数。

• global rank（全局编号）：0 到 world_size-1 之间的整数，唯一标识进程组中的 worker。例如 world size 为 2 时，一个进程为 rank 0（主进程），另一个为 rank 1。

• local world size：跨节点运行时，某个节点本地的 worker 数。例如在两个节点上各启动 4 个 worker，全局 world size 为 8，而 local world size 为 4。单节点时，本地数量等于全局数量。

• local rank（本地编号）：0 到 local_world_size-1 之间的整数，唯一标识机器上的本地 worker。例如两个节点各启动 4 个进程，则每个节点的 local rank 都是 0、1、2、3。单节点多进程程序中，local rank 与 global rank 相同。



原文第 32 页 · 原式 / 代码 / 图示

图 3：两个节点、world size 为 8 的分布式程序。每个工作进程都有全局编号（0-7）和本地编号（0-3）。图片来源：https://lightning.ai/docs/fabric/stable/advanced/distributed_communication.html 。

5.1.1 分布式应用基准测试的最佳实践

这一部分会反复测量分布式程序，以理解通信开销。建议遵循：

• 尽可能在同一台机器上测试，以便进行受控比较。

• 正式计时前先预热数步，对 NCCL 通信尤其重要。通常预热 5 次即可。

• 在 GPU 上测试时，调用 torch.cuda.synchronize() 等待 CUDA 操作完成。即使通信操作设置 async_op=False，也需要这样做，因为该调用只保证通信操作已排入 GPU 队列，并不意味着通信已经完成。[脚注 3：https://github.com/pytorch/pytorch/issues/68112#issuecomment-965932386]

• 不同 rank 的耗时可能略有差异，通常汇总各 rank 的测量以改善估计。all-gather，特别是 dist.all_gather_object，可用于收集所有 rank 的结果。

• 一般先在本地 CPU 上用 Gloo 调试，再按题目要求在 GPU 上用 NCCL 测试。切换时只需修改 init_process_group 调用和张量设备转换。

题目 distributed_communication_single_node：单节点分布式通信（5 分）

编写脚本，测试单节点、多进程环境下 all-reduce 的运行时间。上面的示例可以作为起点。改变以下设置：

• all-reduce 数据量：float32 张量，大小分别为 1MB、10MB、100MB、1GB。

• GPU / 进程数量：2、4 或 6。

资源要求：最多 6 张 GPU，每次基准测试应在 5 分钟内完成。

提交内容：用图和 / 或表比较不同设置，并用 2-3 句话评论结果，以及你对各因素相互影响的理解。

5.2 分布式数据并行训练的朴素实现

了解 PyTorch 分布式程序的基础后，下面实现一个最简分布式数据并行（DDP）训练版本。

数据并行将一个批次拆分到多个设备上，例如多个 GPU，使训练能够使用单个设备放不下的大批次。例如四个设备分别最多处理批大小 32，那么数据并行可实现有效批大小 128。

朴素 DDP 的流程如下。首先，各设备构造一个随机初始化模型，再用 broadcast 集合通信把 rank 0 的参数发送给所有其他 rank。训练开始时，各设备拥有完全相同的模型参数和优化器状态副本，例如 Adam 累积的梯度统计量。

1. 给定含 n 个样本的批次，将其分片，每个设备分得 n/d 个互不重叠的样本，d 为数据并行设备数量。d 应整除 n，否则不同 rank 工作量不均，训练步骤会受最慢者限制。

2. 各设备使用本地参数副本，对分得的 n/d 个样本执行前向和反向，计算本地梯度。此时梯度只对应本设备收到的样本。

3. 使用 all-reduce 对不同设备的梯度求平均，使每个设备都持有针对全部 n 个样本的平均梯度。

4. 各设备执行优化器更新，修改自己的参数副本。从优化器角度看，它只是在优化本地模型。由于所有设备初始参数、优化器状态相同，每次又使用同样的平均梯度，因此参数和优化器状态会持续保持同步。至此完成一次迭代，随后重复。

题目 naive_ddp：朴素 DDP（5 分）

提交内容：实现朴素分布式数据并行训练，在反向传播结束后，逐个对参数梯度执行 all-reduce。为测试实现，完成 adapters.get_ddp，以及可选的 adapters.ddp_on_after_backward，再运行 uv run pytest tests/test_ddp.py。

题目 naive_ddp_benchmarking：朴素 DDP 基准测试（3 分）

该实现中，每次反向后，逐个对参数梯度跨 rank 进行 all-reduce。为了理解数据并行训练的开销，编写脚本，用朴素 DDP 训练之前实现的语言模型，测量每个训练步骤的总时间及梯度通信所占比例。在单节点、2 张 GPU 的环境中，使用第 2.1.2 节的 xl 模型收集数据。

提交内容：描述基准测试设置，并报告每种设置下单次训练迭代的耗时和梯度通信耗时。

5.3 改进最简 DDP 实现

第 5.2 节的最简实现有两个主要限制：

1. 每个参数张量单独执行一次 all-reduce。每次通信调用都会产生开销，因此合并通信可能降低开销。

2. 等到整个反向传播结束后才通信。但梯度是逐步计算出来的，一个参数的梯度一旦就绪就可以立即通信，不必等其他梯度。这可以让梯度通信与反向计算重叠，从而减少 DDP 开销。

本部分将分别解决这两个问题，并测量对训练速度的影响。

5.3.1 减少通信调用次数

不再为每个参数张量发起一次通信，而尝试批量执行 all-reduce：把需要归约的梯度拼成一个张量，再跨所有 rank 归约该合并张量。torch._utils._flatten_dense_tensors 和 torch._utils._unflatten_dense_tensors 可能有所帮助。

题目 minimal_ddp_flat_benchmarking：梯度展平的最简 DDP 基准测试（2 分）

修改最简 DDP，使其通信对象为全部参数梯度展平后组成的单个张量。在之前的条件下，即单节点、2 张 GPU、第 2.1.2 节的 xl 模型，比较它与逐参数 all-reduce 的版本。

提交内容：报告单次批量 all-reduce 的 DDP 每轮训练耗时和梯度通信耗时，并用 1-2 句话比较合并通信与逐参数通信的结果。

5.3.2 将计算与逐参数梯度通信重叠

合并调用虽然可能降低大量小 all-reduce 的调用开销，但通信时间仍会直接增加总开销。反向传播从损失向输入逐层计算梯度，因此可以在梯度就绪后立即归约，让反向计算与梯度通信重叠。

先实现一个 DDP 包装器，在每个参数梯度于反向中就绪时，异步发起该参数的 all-reduce，并测试性能。以下提示可能有帮助。

反向钩子：可使用 register_post_accumulate_grad_hook，在参数梯度于反向传播中完成累积之后，自动调用某个函数。[脚注 4：用法参见 https://pytorch.org/docs/stable/generated/torch.Tensor.register_post_accumulate_grad_hook.html 。]

异步通信：所有 PyTorch 集合通信都支持同步执行（async_op=False）和异步执行（async_op=True）。同步调用阻塞到集合操作被排入 GPU 队列为止，但 CUDA 操作是异步的，因此不代表执行已经完成。尽管如此，后续使用其输出的函数仍能按预期工作。

相比之下，异步调用会返回一个分布式请求句柄：函数返回时，不能保证通信已经排入 GPU 队列，更不能保证完成。对返回句柄调用 handle.wait()，可等待操作排入 GPU 队列，从而使后续操作可以安全使用其输出。

[脚注 5] 在使用多个 CUDA stream 的高级场景中，可能需要显式跨流同步，以确保后续操作能够使用输出。参见 https://pytorch.org/docs/stable/notes/cuda.html#cuda-streams 。

以下两段分别通过同步和异步方式，对一个张量列表中的每个张量执行 all-reduce：

```
tensors = [torch.rand(5) for _ in range(10)]

# Synchronous, block until operation is queued on the GPU.
for tensor in tensors:
    dist.all_reduce(tensor, async_op=False)

# Asynchronous, return immediately after each call and
# wait on results at the end.
handles = []
for tensor in tensors:
    handle = dist.all_reduce(tensor, async_op=True)
    handles.append(handle)

# ...
# Possibly execute other commands that don't rely on the all_reduce results
# ...

# Ensure that all-reduce calls were queued and
# therefore other operations depending on the
# all-reduce output can be queued.
for handle in handles:
    handle.wait()
handles.clear()
```

原文第 35 页 · 原式 / 代码 / 图示

题目 ddp_overlap_individual_parameters：逐参数通信与计算重叠的 DDP（5 分）

实现一个处理 DDP 训练的 Python 类，包装任意 PyTorch nn.Module，负责训练前广播权重以统一各 rank 的初始参数，并发起梯度平均通信。建议公开接口如下：

• def __init__(self, module: torch.nn.Module)：接收已实例化、待并行化的模块，构造负责跨 rank 梯度同步的 DDP 容器。

• def forward(self, *inputs, **kwargs)：将给定位置和关键字参数转交给被包装模块的 forward()。

• def finish_gradient_synchronization(self)：被调用时等待异步通信，使 GPU 上相关操作完成必要同步。

用该类训练时，先传入模块进行包装，再在 optimizer.step() 之前调用 finish_gradient_synchronization()，确保依赖梯度的优化器更新可以安全排入执行队列：

```
model = ToyModel().to(device)
ddp_model = DDP(model)

for _ in range(train_steps):
    x, y = get_batch()
    logits = ddp_model(x)
    loss = loss_fn(logits, y)
    loss.backward()
    ddp_model.finish_gradient_synchronization()
    optimizer.step()
```

原文第 36 页 · 原式 / 代码 / 图示

提交内容：实现 DDP 容器类，使梯度通信与反向计算重叠。先实现 adapters.get_ddp，以及 adapters.ddp_on_after_backward；后者可选，具体取决于实现。再运行 uv run pytest tests/test_ddp.py。建议多次运行，例如 5 次，以确保稳定通过。

题目 ddp_overlap_individual_parameters_benchmarking：逐参数重叠 DDP 基准测试（1 分）

(a) 测量让反向计算与逐参数梯度通信重叠后的 DDP 性能。在相同设置（单节点、2 张 GPU、xl 模型）下，与之前两种最简 DDP 比较：逐参数 all-reduce，以及把全部参数梯度拼接后进行单次 all-reduce。

提交内容：报告重叠实现每次训练迭代耗时，并用 1-2 句话比较结果。

(b) 在相同设置下，用 Nsight 分析基准代码，对比初始 DDP 和重叠实现。直观比较两条轨迹，提供截图证明一个实现将通信与计算重叠，而另一个没有。

提交内容：两张截图，分别来自初始 DDP 和重叠 DDP，直观展示通信是否与反向传播重叠。

6 优化器状态分片

DDP 概念简单，而且通常效果很好，但每个 rank 都要保存一套模型参数和优化器状态，冗余会带来显著内存成本。例如 AdamW 为每个参数维护两个浮点数，因此优化器状态占用的内存为模型权重的两倍。

S. Rajbhandari 等 [5] 提出了几种消除这类冗余的方法：跨 rank 分片 (1) 优化器状态、(2) 梯度和 (3) 参数，在需要时通过通信交换。

本部分实现简化的优化器状态分片，以降低每个 rank 的内存使用。每个 rank 的优化器只负责一部分参数，约为 1/world_size，而不再为全部参数保存状态。每次更新时，只更新本分片负责的参数，然后把更新后的参数广播给其他 rank，确保每次优化器更新后模型参数仍然同步。

题目 optimizer_state_sharding：优化器状态分片（15 分）

实现一个处理优化器状态分片的 Python 类，包装任意 PyTorch optim.Optimizer，并在每次更新后同步参数。建议公开接口如下。

• def __init__(self, params, optimizer_cls: Type[Optimizer], **kwargs: Any)：初始化分片状态优化器。params 是待优化的参数集合，也可以是参数组，例如用户希望对模型不同部分使用不同学习率时。这些参数会跨所有 rank 分片。optimizer_cls 指定被包装优化器的类型，如 optim.AdamW；其余关键字参数转交给 optimizer_cls 的构造函数。此方法中务必调用 torch.optim.Optimizer 父类构造函数。

• def step(self, closure, **kwargs)：以给定 closure 和关键字参数调用被包装优化器的 step()；参数更新后，与其他 rank 同步。

• def add_param_group(self, param_group: dict[str, Any])：向分片优化器添加参数组。父类构造函数会在初始化时调用它，训练过程中也可能调用，例如逐步解冻模型层。因此该方法需要负责在各 rank 间分配模型参数。

提交内容：实现优化器状态分片容器类。先实现 adapters.get_sharded_optimizer，再运行 uv run pytest tests/test_sharded_optimizer.py。建议多次运行，例如 5 次，以确保稳定通过。

实现之后，分析它对训练内存峰值与运行开销的影响。

题目 optimizer_state_sharding_accounting：优化器状态分片的开销核算（5 分）

(a) 编写脚本，比较语言模型训练启用和不启用优化器状态分片时的内存峰值。使用标准配置：单节点、2 张 GPU、xl 模型。报告模型初始化之后、优化器更新紧前和紧后的峰值内存。结果是否符合预期？拆解各设置下内存的组成，例如参数、优化器状态等分别占多少。

提交内容：用 2-3 句话报告内存峰值，并分解模型与优化器各组件的内存占用。

(b) 该分片实现如何影响训练速度？在标准配置下，分别测量使用和不使用分片时的单次迭代耗时。

提交内容：用 2-3 句话回答并报告耗时。

(c) 我们的分片方式与 ZeRO stage 1 有何不同？文献 [5] 将后者称为 ZeRO-DP P_os。

提交内容：用 2-3 句话概括差异，尤其是内存和通信量方面。

7 全分片数据并行

通过优化器状态分片与数据并行，我们已经可以沿数据并行轴拆分优化器状态和激活。但模型权重仍然重复存储，每张 GPU 都有完整副本。

将数据并行（DP）轴转为全分片数据并行（FSDP）轴即可解决。FSDP 中，每张 GPU 只保存每个权重张量的一片；在前向或反向计算前，需通过 all-gather 从其他 GPU 拉取其余分片，重建完整权重。

为避免 GPU 等待通信，多数 FSDP 实现会提前调度各层的 all-gather，使权重在使用前就绪，不让通信阻塞计算。只要通信速度跟得上计算，且调度得当，权重分片通信就能离开关键路径，不增加整体耗时。

有些层的内存和计算规模很小，通信延迟开销不值得，因此应标记为不进行 FSDP 分片。在我们的架构中，这主要是归一化层；剩下要分片的是嵌入层和所有线性层。

主权重（master weights）需要以 FP32 保存，因为反复累加更新的数值对精度敏感；但计算时不一定要使用 FP32。在混合精度中，权重使用前总要转换为低精度计算类型，因此可以在通信前就转换，以节约带宽。

题目 fsdp：全分片数据并行（15 分）

实现 FSDP 的 Python 类，包装任意 PyTorch nn.Module（整个模型），并对其中所有 Linear 或 Embedding 层注册钩子或进行包装。建议公开接口如下。

• def __init__(self, module: torch.nn.Module, compute_dtype: torch.dtype | None = None)：接收已实例化、待并行化的模型，构造负责权重 all-gather 和梯度 reduce-scatter 的 FSDP 模块。钩子或层包装器应及时收集权重，以供前向使用。为限制内存，只有在目标层之前第二层完成前向后，才开始收集目标层权重。反向中也应通过 all-gather 准备计算所需权重；梯度就绪后，将其 reduce-scatter 到相应 rank。使用后务必释放已聚合的完整权重。指定 compute_dtype 时，通信和计算前把权重转成该类型，但主权重和优化器更新保持 FP32。

• def forward(self, *inputs, **kwargs)：将给定位置和关键字参数转交给被包装模块的 forward()。

• def finish_gradient_synchronization(self)：被调用时等待异步通信在 GPU 上完成必要同步。

提交内容：实现 FSDP 容器类，每个分片应兼容作业 1 的标准 AdamW。实现 adapters.get_fsdp，然后运行 uv run pytest tests/test_fsdp.py。建议多次运行，例如 5 次，以发现竞态条件。

题目 fsdp_accounting：FSDP 开销核算（5 分）

(a) 根据第 6 节分析，实现 FSDP 后，预计可以从内存峰值中节约多少？计算时可忽略每张 GPU 用于 all-gather 权重的预分配缓冲区。

提交内容：用 2-3 句话给出结论。

(b) 在两张 GPU 上分析 xl 模型，重点观察权重 all-gather。通信能否在前向需要之前完成？

提交内容：用 2-3 句话回答并给出耗时，附 Nsight 截图支持结论。

8 并行策略分析

训练还可以沿其他轴并行。常见策略包括：

• 数据并行（DP）：批次数据拆分到多个设备，各设备为本地批次计算梯度，再跨设备求平均。

• 全分片数据并行（FSDP）：在数据并行之上，还把优化器状态、梯度和权重分片，以减少内存。前向和反向时，各设备需要从其他设备聚合权重分片。

• 张量并行（TP）：沿输入或输出维度切分权重矩阵。各设备计算本分片对应的激活，再跨设备归约或聚合激活。

• 流水线并行（PP）：按层把模型划分为多个阶段，各阶段运行在不同设备上。

• 专家并行（EP）：将混合专家模型的专家分配到不同设备，每个设备计算自己负责专家的输出。

本节在简化设置中通过基本数学计算，判断如何选择和组合并行策略。重点讨论 DP、FSDP、TP 及其组合。方法是计算各策略通信成本，并与计算成本比较，从而确定在通信成为瓶颈之前能扩展到多少设备。

若想深入了解 TPU/GPU 拓扑和并行策略，TPU Scaling Book（J. Austin 等 [6]）是很好的资源。更详细的流水线并行讨论可见 The Ultra-Scale Playbook 的附录 [7]。这两份资料的其他部分也包含许多有用信息。对应链接：https://jax-ml.github.io/scaling-book/training/ ；https://nanotron-ultrascale-playbook.static.hf.space/#pp_communication_analysis 。

8.1 通信原语

先理解通信原语。在简化设置中，假设有 N 个设备，编号为 0,…,N-1，每对设备之间都有链路。每个设备的出口带宽为 W，即可以每秒 W 字节的速率向其他设备发送数据。如何实现 gather 和 reduce？

all-gather 的常见实现是环形 all-gather。开始时，设备 i 持有大小为 S/N 的块 x_i；结束时，每个设备都持有总大小为 S 字节的完整张量 x=[x_0,…,x_(N-1)]。将设备排成环，每步各设备向右侧设备发送当前块，并保存左侧设备发来的块；下一步继续将刚收到的块向右传递，同时从左侧接收新块。经过 N-1 步，各设备都拥有完整张量。

在理想设置中，每步各设备同时以带宽 W 发送大小 S/N 的块，共 N-1 步，因此环形 all-gather 耗时为 ((N-1)/N)·(S/W) 秒。

接着分析环形 reduce-scatter。开始时，各设备 i 持有大小 S 的完整张量 x^(i)。目标是计算 y=Σ_i x^(i)，但结束时设备 i 只持有大小 S/N 的结果块 y_i。仍将设备排成环，并把各自 x^(i) 分成 N 个大小 S/N 的块。传递方式类似 all-gather，不过设备在继续转发前，要先把自己的对应贡献加到正在传递的部分和中。具体为：

对于步骤 t=1,…,N-1，设备 i 执行：

• 若 t=1，初始化 y←x^(i)，用来保存当前部分和。

• 将 y_((i-t) mod N) 发送到设备 (i+1) mod N。

• 从设备 (i-1) mod N 接收 z_((i-t-1) mod N)。

• 更新部分和：y_((i-t-1) mod N)←y_((i-t-1) mod N)+z_((i-t-1) mod N)。

经过 N-1 步，设备 i 得到结果块 y_i 的完整总和。因此环形 reduce-scatter 同样耗时 ((N-1)/N)·(S/W) 秒。

最后实现环形 all-reduce：各设备 i 从大小 S 的完整张量 x^(i) 出发，最终都得到 y=Σ_i x^(i)。先执行环形 reduce-scatter，再执行环形 all-gather，总耗时为 2·((N-1)/N)·(S/W) 秒。

题目 alternate_ring_all_reduce：另一种环形 all-reduce（1 分）

不用“环形 reduce-scatter 后接环形 all-gather”，改用以下算法。

对于步骤 t=1,…,N-1，设备 i 执行：

• 若 t=1，初始化 y←x^(i)，保存当前部分和。

• 将 x^((i-t+1) mod N) 发送到设备 (i+1) mod N。

• 从设备 (i-1) mod N 接收 x^((i-t) mod N)。

• 更新部分和：y←y+x^((i-t) mod N)。

在同样设置下，每个设备出口带宽 W，每个 x^(i) 大小 S，该算法需要多长时间？

提交内容：用 S、N、W 表示的答案，以及一句理由。

8.2 数据并行分析

有了通信原语，就可以分析并行策略。考虑单个 FFN 层的并行化。给定输入 x，前向如下：

$$\boldsymbol{x_1} = \boldsymbol{x}\boldsymbol{W_1} \tag{20}$$
$$\boldsymbol{x_2} = \boldsymbol{x}\boldsymbol{W_2} \tag{21}$$
$$\boldsymbol{z} = f(\boldsymbol{x_1}) * \boldsymbol{x_2} \tag{22}$$
$$\boldsymbol{y} = \boldsymbol{z}\boldsymbol{W_3}, \tag{23}$$

原文第 41 页 · 原式 / 代码 / 图示

其中 x 形状为 (B,D)，W_1、W_2 为 (D,D_FF)，W_3 为 (D_FF,D)；f 为逐元素激活函数，例如 SiLU；* 表示逐元素乘法。

明确写出反向也很有帮助。给定形状为 (B,D) 的 dy，反向为：

$$\boldsymbol{dz} = \boldsymbol{dy}\boldsymbol{W_3}^\top \tag{24}$$
$$\boldsymbol{dx_2} = \boldsymbol{dz} * f(\boldsymbol{x_1}) \tag{25}$$
$$\boldsymbol{dx_1} = \boldsymbol{dz} * f'(\boldsymbol{x_1}) * \boldsymbol{x_2} \tag{26}$$
$$\boldsymbol{dx} = \boldsymbol{dx_1}\boldsymbol{W_1}^\top + \boldsymbol{dx_2}\boldsymbol{W_2}^\top \tag{27}$$
$$\boldsymbol{dW_3} = \boldsymbol{z}^\top \boldsymbol{dy} \tag{28}$$
$$\boldsymbol{dW_2} = \boldsymbol{x}^\top \boldsymbol{dx_2} \tag{29}$$
$$\boldsymbol{dW_1} = \boldsymbol{x}^\top \boldsymbol{dx_1}, \tag{30}$$

原文第 41 页 · 原式 / 代码 / 图示

这里 * 仍表示逐元素乘法。

在 N_DP 个设备的数据并行中，输入 x 沿批次维分片，每片 x^(i) 形状为 (B/N_DP,D)。前向照常计算，不需要集合通信，产生同样按批次分片的 y^(i)，形状为 (B/N_DP,D)。反向使用分片激活照常计算，设备 i 得到：

$$d\boldsymbol{W}_{\boldsymbol{3}}^{(\boldsymbol{i})} = \boldsymbol{z}^{(\boldsymbol{i})\top} d\boldsymbol{y}^{(\boldsymbol{i})} \tag{31}$$

$$d\boldsymbol{W}_{\boldsymbol{2}}^{(\boldsymbol{i})} = \boldsymbol{x}^{(\boldsymbol{i})\top} d\boldsymbol{x}_{\boldsymbol{2}}^{(\boldsymbol{i})} \tag{32}$$

$$d\boldsymbol{W}_{\boldsymbol{1}}^{(\boldsymbol{i})} = \boldsymbol{x}^{(\boldsymbol{i})\top} d\boldsymbol{x}_{\boldsymbol{1}}^{(\boldsymbol{i})}, \tag{33}$$

原文第 42 页 · 原式 / 代码 / 图示

这里不再对全部 B 个外积求和，而只包含本设备输入分片对应的 B/N_DP 个外积的部分和。因此，需要跨设备 all-reduce，才能得到完整的 dW_3、dW_2、dW_1。

题目 data_parallel_calcs：数据并行计算（3 分）

现在计算何时数据并行会受通信限制。C 表示设备加速器的计算速度，单位 FLOP/s；W 表示每个设备的出口带宽，单位字节/秒。由此可求计算时间和通信时间。由于二者可以重叠，通信时间大于计算时间时，通信成为瓶颈。假设全部权重和激活为 FP16，即每个元素 2 字节。

(a) 使用 N_DP 路数据并行，反向需要多少 FLOPs？可忽略所有非矩阵乘法操作。回顾形状为 (A,B)·(B,C)→(A,C) 的矩阵乘法需要 2ABC 次浮点运算。

提交内容：用 B、D、D_FF、N_DP 表示的答案，以及一句理由。

(b) 使用 N_DP 路数据并行，反向需要多少通信时间？

提交内容：用 B、D、D_FF、N_DP、W 中适用的变量表示，并给出一句理由。

(c) 其他参数固定时，N_DP 最大能增长到多少，才不会受通信瓶颈限制？

提交内容：一个不等式，一侧为 N_DP，另一侧是由 B、D、D_FF、C、W 中适用变量构成的表达式，并给出一句理由。

8.3 全分片数据并行分析

接着分析 FSDP。与 DP 一样，它沿批次维拆分输入和激活；此外，为节约内存，还拆分优化器状态、梯度和权重。权重可沿任一维切分，在设备 i 上形成 W_1^(i)、W_2^(i)、W_3^(i)，每片均含 DD_FF/N_FSDP 个元素。

前向仍在按批次分片的输入上执行数据并行计算，但必须先跨设备 all-gather 权重分片：

$$\boldsymbol{W}_{\boldsymbol{1}} = \text{all-gather}\left(\left\{\boldsymbol{W}_{\boldsymbol{1}}^{(\boldsymbol{i})}\right\}_{i=0}^{N_{\text{FSDP}}-1}\right) \tag{34}$$

$$\boldsymbol{W}_{\boldsymbol{2}} = \text{all-gather}\left(\left\{\boldsymbol{W}_{\boldsymbol{2}}^{(\boldsymbol{i})}\right\}_{i=0}^{N_{\text{FSDP}}-1}\right) \tag{35}$$

原文第 42 页 · 原式 / 代码 / 图示

$$\boldsymbol{W}_{\boldsymbol{3}} = \text{all-gather}\left(\left\{\boldsymbol{W}_{\boldsymbol{3}}^{(\boldsymbol{i})}\right\}_{i=0}^{N_{\text{FSDP}}-1}\right) \tag{36}$$

$$(\text{do batch-sharded forward pass}) \tag{37}$$

原文第 43 页 · 原式 / 代码 / 图示

式 (37) 的文字表示“执行按批次分片的前向传播”。

all-gather 的调度时间有一定自由度：只需在权重使用前完成聚合，并在使用后立即丢弃完整权重以降低内存。本节为简洁起见，将三个 all-gather 列在一起。

反向同样先聚合权重，再执行数据并行的反向计算。但每个设备只需要自己权重分片的梯度，不再需要 all-reduce，而改为 reduce-scatter：

$$\boldsymbol{W_1} = \text{all-gather}\left(\left\{\boldsymbol{W_1^{(i)}}\right\}_{i=0}^{N_{\text{FSDP}}-1}\right) \tag{38}$$

$$\boldsymbol{W_2} = \text{all-gather}\left(\left\{\boldsymbol{W_2^{(i)}}\right\}_{i=0}^{N_{\text{FSDP}}-1}\right) \tag{39}$$

$$\boldsymbol{W_3} = \text{all-gather}\left(\left\{\boldsymbol{W_3^{(i)}}\right\}_{i=0}^{N_{\text{FSDP}}-1}\right) \tag{40}$$

$$(\text{do batch-sharded backward pass}) \tag{41}$$

$$\boldsymbol{dW_1^{(i)}} = \text{reduce-scatter}\left(\left\{\boldsymbol{dW_1^{(i)}}\right\}_{i=0}^{N_{\text{FSDP}}-1}\right) \tag{42}$$

$$\boldsymbol{dW_2^{(i)}} = \text{reduce-scatter}\left(\left\{\boldsymbol{dW_2^{(i)}}\right\}_{i=0}^{N_{\text{FSDP}}-1}\right) \tag{43}$$

$$\boldsymbol{dW_3^{(i)}} = \text{reduce-scatter}\left(\left\{\boldsymbol{dW_3^{(i)}}\right\}_{i=0}^{N_{\text{FSDP}}-1}\right) \tag{44}$$

原文第 43 页 · 原式 / 代码 / 图示

式 (41) 的文字表示“执行按批次分片的反向传播”。注意，上标 (i) 的分片记号在这里有两种含义：reduce-scatter 的输入是针对完整权重、只累加了部分样本的梯度；输出则是针对权重分片、已经累加全部样本的梯度。

题目 fsdp_calcs：全分片数据并行计算（3 分）

在与数据并行相同的设置下，计算 FSDP 何时成为通信瓶颈。

(a) 使用 N_FSDP 路 FSDP，反向需要多少 FLOPs？前向呢？

提交内容：两个用 B、D、D_FF、N_FSDP 表示的答案，各附一句理由。

(b) 使用 N_FSDP 路 FSDP，反向需要多少通信时间？前向呢？

提交内容：两个由 B、D、D_FF、N_FSDP、W 中适用变量表示的答案，各附一句理由。

(c) 其他参数固定时，N_FSDP 最大能增长到多少，才不会在反向中受通信限制？前向呢？

提交内容：两个不等式，一侧为 N_FSDP，另一侧为 B、D、D_FF、C、W 中适用变量构成的表达式，各附一句理由。

8.4 张量并行分析

实践中，FSDP 常与张量并行（TP）结合。TP 将每个权重矩阵沿输入维或输出维跨设备切分。沿输入维切分通常称为“行并行”，沿输出维切分通常称为“列并行”。

假设要并行计算 xW，其中 x 形状为 (B,D)，W 为 (D,D_FF)。列并行中，每个 W^(i) 分片形状为 (D,D_FF/N_TP)，有：

$$\boldsymbol{x}\boldsymbol{W} = \text{all-gather}\left(\left\{\boldsymbol{x}\boldsymbol{W}^{(i)}\right\}_{i=0}^{N_{\text{TP}}-1}\right). \tag{45}$$

原文第 44 页 · 原式 / 代码 / 图示

行并行中，每个 W^(i) 形状为 (D/N_TP,D_FF)，各设备在矩阵乘法前也将输入 x 切为形状 (B,D/N_TP) 的 x^(i)。于是：

$$\boldsymbol{x}\boldsymbol{W} = \text{all-reduce}\left(\left\{\boldsymbol{x}^{(i)}\boldsymbol{W}^{(i)}\right\}_{i=0}^{N_{\text{TP}}-1}\right). \tag{46}$$

原文第 44 页 · 原式 / 代码 / 图示

并行 FFN 时，使用如下配置：W_1 与 W_2 列并行，即沿输出维分片；W_3 行并行，即沿输入维分片。由于行并行权重只需要输入的一片，这种安排可省去列并行之后的 all-gather。给定 (B,D) 的输入 x，前向如下：

$$\boldsymbol{x}_{\mathbf{1}}^{(i)} = \boldsymbol{x}\boldsymbol{W}_{\mathbf{1}}^{(i)} \tag{47}$$

$$\boldsymbol{x}_{\mathbf{2}}^{(i)} = \boldsymbol{x}\boldsymbol{W}_{\mathbf{2}}^{(i)} \tag{48}$$

$$\boldsymbol{z}^{(i)} = f\left(\boldsymbol{x}_{\mathbf{1}}^{(i)}\right) * \boldsymbol{x}_{\mathbf{2}}^{(i)} \tag{49}$$

$$\boldsymbol{y}^{(i)} = \boldsymbol{z}^{(i)}\boldsymbol{W}_{\mathbf{3}}^{(i)} \tag{50}$$

$$\boldsymbol{y} = \text{all-reduce}\left(\left\{\boldsymbol{y}^{(i)}\right\}_{i=0}^{N_{\text{TP}}-1}\right), \tag{51}$$

原文第 44 页 · 原式 / 代码 / 图示

其中 W_1^(i)、W_2^(i) 形状为 (D,D_FF/N_TP)，W_3^(i) 形状为 (D_FF/N_TP,D)。

题目 tp_calcs：张量并行计算（4 分）

在与 DP 和 FSDP 相同的设置下，计算 TP 何时受通信限制。

(a) 给定形状 (B,D) 的 dy，写出上述 TP 策略的反向传播，其中 W_1^(i)、W_2^(i) 为 (D,D_FF/N_TP)，W_3^(i) 为 (D_FF/N_TP,D)。

提交内容：一组描述反向的方程，使用 dy、分片权重 W_1^(i)、W_2^(i)、W_3^(i)，前向保存的激活 x、x_1^(i)、x_2^(i)、z^(i)、y^(i)，通信原语，以及自行定义的中间变量。方程应得到每个设备的梯度 dW_1^(i)、dW_2^(i)、dW_3^(i)，以及反向输出 dx。可以参考并修改第 8.2 节的非分片反向公式。

(b) 使用 N_TP 路 TP，前向需要多少 FLOPs？反向呢？

提交内容：两个用 B、D、D_FF、N_TP 表示的答案，各附一句理由。

(c) 使用 N_TP 路 TP，前向需要多少通信时间？反向呢？

提交内容：两个由 B、D、D_FF、N_TP、W 中适用变量表示的答案，各附一句理由。

(d) 其他参数固定时，N_TP 最大能增长到多少，才不会在反向中受通信限制？前向呢？

提交内容：两个不等式，一侧为 N_TP，另一侧为 B、D、D_FF、C、W 中适用变量构成的表达式，各附一句理由。

8.5 二维并行：FSDP + TP

现在终于可以组合并行策略了。本节研究如何最佳地结合 FSDP 和 TP。提示：前面的推导应该表明，批大小和模型规模限制了设备扩展数量；增大这些量，可继续增加设备而不受通信限制。

但批大小超过一定程度后，模型效果会下降，因为梯度噪声明显减小，SGD 的隐式正则化作用随之减弱；这一阈值常称为“临界批大小”。而缩放定律通常会告诉我们模型应做多大，这也是下一次作业的任务。

本节考虑一个简化情景：批大小、模型规模、带宽、加速器速度等参数都已给定。你的任务是选取 FSDP 和 TP 配置，扩展到尽可能多的设备，同时保持计算受限，而不是通信受限。

先说明组合机制。每个设备都有 TP 编号 i=0,…,N_TP-1 和 FSDP 编号 j=0,…,N_FSDP-1，构成二维网格，总设备数 N=N_TP·N_FSDP。

先按 TP 规则，将 W_1、W_2 沿输出维分片，W_3 沿输入维分片，因此要对激活执行 TP 形式的 all-reduce。接着加入 FSDP：沿输入批次维分片，并沿尚未被 TP 切分的那个维度进一步切分每个权重矩阵。因此，在 TP 形式的前向或反向之前，需要对权重执行 FSDP 形式的 all-gather；在 TP 反向之后，对权重梯度执行 reduce-scatter。

最终，设备 (i,j) 持有 W_1^(i,j)、W_2^(i,j)，形状为 (D/N_FSDP,D_FF/N_TP)，以及 W_3^(i,j)，形状为 (D_FF/N_TP,D/N_FSDP)。给定按批次分片的 x^(j)，形状为 (B/N_FSDP,D)，前向可写为：

$$\boldsymbol{W_1^{(i)}} = \text{all-gather}\left(\left\{\boldsymbol{W_1^{(i,j)}}\right\}_{j=0}^{N_{\text{FSDP}}-1}\right) \tag{52}$$

$$\boldsymbol{W_2^{(i)}} = \text{all-gather}\left(\left\{\boldsymbol{W_2^{(i,j)}}\right\}_{j=0}^{N_{\text{FSDP}}-1}\right) \tag{53}$$

$$\boldsymbol{W_3^{(i)}} = \text{all-gather}\left(\left\{\boldsymbol{W_3^{(i,j)}}\right\}_{j=0}^{N_{\text{FSDP}}-1}\right) \tag{54}$$

$$\boldsymbol{x_1^{(i,j)}} = \boldsymbol{x^{(j)}}\boldsymbol{W_1^{(i)}} \tag{55}$$

$$\boldsymbol{x_2^{(i,j)}} = \boldsymbol{x^{(j)}}\boldsymbol{W_2^{(i)}} \tag{56}$$

原文第 45 页 · 原式 / 代码 / 图示

$$\boldsymbol{z^{(i,j)}} = f\left(\boldsymbol{x_1^{(i,j)}}\right) * \boldsymbol{x_2^{(i,j)}} \tag{57}$$

$$\boldsymbol{y^{(i,j)}} = \boldsymbol{z^{(i,j)}}\boldsymbol{W_3^{(i)}} \tag{58}$$

$$\boldsymbol{y^{(j)}} = \text{all-reduce}\left(\left\{\boldsymbol{y^{(i,j)}}\right\}_{i=0}^{N_{\text{TP}}-1}\right), \tag{59}$$

原文第 46 页 · 原式 / 代码 / 图示

最终得到按批次分片的 y^(j)，形状为 (B/N_FSDP,D)。为简洁起见，本节省略反向，只讨论前向；不过到此你应已具备自行写出反向所需的全部信息。

题目 fsdp_tp_calcs：二维并行计算（6 分）

在前述相同设置下，计算二维并行何时受通信限制。

(a) 使用 N_FSDP 路 FSDP 与 N_TP 路 TP，前向需要多少 FLOPs？

提交内容：用 B、D、D_FF、N_FSDP、N_TP 表示的答案，以及一句理由。

(b) 同样配置下，前向需要多少通信时间？假设两个轴上的通信可以重叠，即 FSDP 轴与 TP 轴的集合通信可相互重叠。

提交内容：用 B、D、D_FF、N_FSDP、N_TP、W 中适用变量表示，并给出一句理由。提示：由于可重叠，应表达为 FSDP 与 TP 两项集合通信成本的最大值 max。

(c) 最优选择 N_TP 和 N_FSDP 时，总设备数 N=N_TP·N_FSDP 最大能增长到多少，才不会使前向受通信瓶颈限制？

提交内容：一个不等式，一侧为 N，另一侧是 B、D、D_FF、C、W 中适用变量构成的表达式，并用几句话和方程解释。

(d) 现在假设 FSDP 轴与 TP 轴的集合通信共享网络资源，不能重叠。最优选择 N_TP 和 N_FSDP 时，N=N_TP·N_FSDP 最大能增长到多少，才不会使前向受通信瓶颈限制？无需考虑将 N_TP、N_FSDP 取整。

提交内容：一个不等式，一侧为 N，另一侧是 B、D、D_FF、C、W 中适用变量构成的表达式，并用几句话和方程解释。

9 排行榜

作业 2 排行榜将测试一个 8B 模型的完整训练步骤速度。你可以发挥创意，用各种技巧优化运行时间和内存，并进行基准测试。

主要限制是不能改变模型的输入 / 输出行为。实现会与 cs336_basics 目录中的模型对照测试。测试输入使用 BF16 并启用因果掩码，必须通过与你普通实现相同的测试。实现也必须由你自己完成，不能使用或复制现有实现。

计时应在两张 B200 GPU 上进行，批大小为 2，序列长度为 32,768。让模型装入显存本身就是刻意设置的难点。完整配置见下方。课程组会核验排名前 5-10 的提交是否正确，并检查性能。用于计时的测试如下：

```
class Config:
    ctx_len = 32768
    vocab_size = 151936
    d_model = 4096
    d_ff = 11008
    num_layers = 34
    num_heads = 32
    torch_dtype = torch.bfloat16
    is_causal = True
    batch_size = 2
cfg = Config()

def test_timing_forward_backward():
    labels, targets = torch.randint(high=cfg.vocab_size, size=(2, cfg.batch_size,
cfg.ctx_len))

    model = BasicsTransformerLM(Config())
    optimizer = AdamW(model.parameters())

    def train_step():
        optimizer.zero_grad(set_to_none=True)
        res = model(labels)
        loss = cross_entropy(res, targets).sum()
        loss.backward()
        optimizer.step()

    timing_results = triton.testing.do_bench(train_step, rep=30_000, warmup=10_000)
    print(timing_results)
```

原文第 47 页 · 原式 / 代码 / 图示

测试过程中，可以缩短重复测量和预热时长；代码中的这两个时长以毫秒为单位。

以下是一些性能优化及降低内存需求的思路：

- 调整内核分块大小，使用 Triton autotune。
- 调整其他 Triton / torch.compile 配置参数。
- 实现融合版 AdamW。

• 基础实现会显式生成完整 logits，形状为 [batch, seq_len, vocab_size]。可以编写融合 LM head 与交叉熵损失的内核，也可以让它立即以融合方式计算反向。

• 改进 FlashAttention：用 Triton 实现反向，而不只是 torch.compile，见第 4.2.3 节。

• 反向对输入遍历两遍：一次计算 dQ，另一次计算 dK、dV，避免原子操作或线程块间同步。

• 因果掩码下提前结束程序实例，跳过确定全为零的分块。

• 将无需掩码的分块与对角线分块分开处理：前者完全不做索引比较，后者只比较一次。

• 在原文所述 Hopper 之后的架构上，按教程中的类似方式使用 TMA（Tensor Memory Accelerator）功能。

• 只有确实需要时，才使用激活检查点，以运行速度换取内存节约。

题目 leaderboard：最快训练步骤排行榜（10 分）

基准测试使用两张 B200，批大小为 2。以完整训练步骤的实际经过时间（wall-clock time）评估，包括前向、损失、反向和 AdamW 更新。

从空的 PyTorch / Triton 缓存开始，整个基准运行必须在 10 分钟内完成，因此要谨慎使用过于激进的 torch.compile 和 Triton 自动调优。

提交内容：包含 AdamW 的完整前向、反向训练步骤所达到的最佳实际耗时。

我们期望排行榜提交优于 10 秒的朴素基线。请在此提交结果：https://github.com/stanford-cs336/assignment2-systems-leaderboard 。

参考文献

以下保留英文题名，便于检索；括号内附中文释义。

[1] T. Dao. “FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning.”（FlashAttention-2：通过更好的并行与工作划分加速注意力。）https://arxiv.org/abs/2307.08691

[2] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, C. Re. “FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.”（FlashAttention：感知 I/O 的快速、内存高效精确注意力。）载于 Advances in Neural Information Processing Systems，A. H. Oh、A. Agarwal、D. Belgrave、K. Cho 编，2022。https://openreview.net/forum?id=H4DqfPSibmx

[3] M. Milakov, N. Gimelshein. “Online normalizer calculation for softmax.”（softmax 归一化因子的在线计算。）https://arxiv.org/abs/1805.02867

[4] H. He. “Making Deep Learning Go Brrrr From First Principles.”（从基本原理出发加速深度学习。）2022。https://horace.io/brrr_intro.html

[5] S. Rajbhandari, J. Rasley, O. Ruwase, Y. He. “ZeRO: Memory Optimizations Toward Training Trillion Parameter Models.”（ZeRO：面向万亿参数模型训练的内存优化。）2020。

[6] J. Austin 等. “How to Scale Your Model.”（如何扩展你的模型。）2025。

[7] H. Z. P. N. M. M. L. W. T. W. Nouamane Tazi Ferdinand Mom. “The Ultra-Scale Playbook: Training LLMs on GPU Clusters.”（超大规模训练手册：在 GPU 集群上训练大语言模型。）2025。

译注：文献 [7] 的作者字段按原 PDF 保留，原文该字段存在不规范的拼接。